CODE READING GUIDE / 2026.10.29

Kokomi Kernel

構造と動作の詳解

Live会話から、身体・世界モデル・記憶まで

何が起きると、どの部品が動くのか。

なぜ、その機能が必要なのか。

現在の実装を、コードの根拠とともに読み解く。

対象: feat/live-conversation-kernel の作業ツリー

HEAD: ad2eb12136eb923a3e97738ddcb36330667f1a30

未コミット・未追跡の実装を含むコード読解資料

日本語 / 図解・動作例・用語集・実装参照付き

この資料の使い方

全体の構造を自分の言葉で説明できるようになることを目的に、現在のコードを入力から出力まで追った資料です。最初から全ファイルを暗記する必要はありません。まず「会話」「身体状態」「操作」「記憶」の流れを分け、必要なときに参照先へ戻ってください。

おすすめの読み順

1周目は01〜03、06〜08、26、31〜33、48を読みます。全体像と責任の境界がつながります。2周目は14〜25と30〜37で、情報の形式と保存を追います。3周目は動作例41〜44をコードと照らし、45以降の一覧・用語集を辞書として使います。

記述の意味

「実装」はコードで確認した動作、「設定」は同梱config.jsonの値、「例」は理解用に組み立てた架空の入力です。「理由」は責任分割や障害対策から読み取れる設計上の意図を説明します。実機で確かめた結果と、コードから分かる結果を混同しません。

対象と調査方法

2026年10月29日（日本時間）の作業フォルダーを調査しました。ソース・設定・プロンプト・既存テストを読み、同梱Node.js 24.22.1で104テストの成功を確認しました。外部APIの有料会話、実機操作、実際の会話ログの内容は検証対象にしていません。秘密情報を含む.envは掲載していません。

参照の使い方

各ページ末尾の「根拠」はリポジトリ内のファイル名と行番号です。HEADだけでは今回のLive実装を再現できないため、末尾の原典一覧と付属のSHA-256一覧を併用してください。左のしおりから各話題へ移動できます。

> 一番大切な区別は、観測・解釈・提案・実行結果です。この四つを分けると、複雑に見える実装をかなり読みやすくできます。

目次

01 現在地：このブランチには何があるか	6
02 全体像：会話と身体を結ぶ二つの経路	7
03 誰が何を決めるか：責任と権限	8
04 ファイル地図：どこを読むと何が分かるか	9
05 起動時：過去を復元してから新しい入力を受ける	10
06 普通に話すとき：雑談はどう流れるか	11
07 身体に関わる依頼：Liveから何を委譲するか	12
08 待ち行列と取り消し：遅い処理をどう扱うか	13
09 会話開始：マイクからセッション確立まで	14
10 会話終了：マイク停止と終了確認は別の段階	15
11 Liveへ渡す身体状態：全部を毎回送らない	16
12 文脈の予算：AIへ何をどれだけ渡すか	17
13 Liveの警告：拒否・無音・音声欠落を分ける	18
14 観測：外から来た情報を共通の封筒に入れる	19
15 状態：観測履歴から「いま」を取り出す	20
16 鮮度：unknown・stale・falseを区別する	21
17 世界モデル：測定値を意味のある事実へ変える	22
18 人物と視覚：見えている人と画像理解は別経路	23
19 接触と音分類：会話以外の入力も意味へ変える	24
20 ドライブ：行動を考えるための必要度	25
21 自発行動：候補を出すことと動くこと	26
22 認知文脈：ChatGPTへ渡す一回分の資料	27
23 ChatGPTブラウザ：API呼出しではない判断経路	28
24 形式監査：AIのJSONをどこで止めるか	29

目次（続き）

25 行動ゲート：提案を実行してよいか判断する 30
26 フィードバック：命令したことと起きたこと 31
27 LEDと涙：何を成功として扱っているか 32
28 公開投稿と人物登録：外部への影響が大きい操作 33
29 情報照会とカメラ：足りない根拠を取りに行く 34
30 会話ログ：断片を原文のまま持つ理由 35
31 記憶とは何か：履歴・要約・採用済み事実 36
32 会話から記憶を作る：抽出と照合の二段階 37
33 自動採用の条件：高い確信度だけでは通らない 38
34 記憶の検索と統合：全部を思い出すわけではない 39
35 記憶処理の復旧：途中で落ちても同じ候補を使う 40
36 社会状態：関係、境界、約束を記録する 41
37 永続化と再起動：何が残り、何が消えるか 42
38 旧音声経路：ASRとTTSを分けていた構成 43
39 旧TTSとfiller：声の出力も状態として扱う 44
40 観測可能性：画面とログで何を追うか 45
41 動作例：LED変更と途中の取り消し 46
42 動作例：暑い部屋とセンサーの停止 47
43 動作例：知らない人、名前の登録、接触 48
44 動作例：会話の断片が一つの記憶になるまで 49
45 HTTPの入口一覧：操作の地図 50
46 設定早見表：現在の数値をどう読むか 51
47 検証：テストで分かること、残ること 52
48 実装の境界：現在できることを過大評価しない 53

目次（続き）

49 コードの読み進め方：一つの出来事から追う 54
50 理解を確かめるための問い 55
51 用語集：観測・世界・認知・身体 56
52 用語集：行動・記憶・確かさ 57
53 用語集：通信・音声・復旧 58
54 実装参照表：観測から実行まで 59
55 実装参照表：Live・記憶・旧音声 60
56 原典・公式資料・この資料の再現条件 61
57 データ例：観測・状態・世界の違い 62
58 データ例：操作の応答と結果を混ぜない 63
59 データ例：記憶の本文・引用・識別子 64
60 なぜこの部品が必要か：設計意図の対応表 65

01 現在地：このブランチには何があるか

このプロジェクトの「カーネル」は、OSの中核プログラムという意味ではありません。心海という身体を持つ対話システムで、センサー、AI、記憶、機体操作をつなぎ、何を事実として保持するかを管理するNode.jsのアプリケーションです。起点はalice.jsです。

調査対象はfeat/live-conversation-kernelの**作業ツリー全体**です。HEADは2026年10月28日のad2eb12136eb923a3e97738ddcb36330667f1a30ですが、Live関連の新規ファイルや既存ファイルの変更は未コミットでした。このPDFの説明は、HEADのコードだけを読んだ場合とは一致しません。

領域	現在の設定・実装	読み取れること
Live会話	enabled=true	通常の音声会話をGPT-Liveが担当する
身体の判断	ChatGPTブラウザ	Puppeteerで既存のログイン済み画面を操作する
旧ASR / TTS / filler	Live中は実効無効	個々のenabled=trueよりLiveの切替条件が優先する
世界モデル・ドライブ	実装あり	観測から規則で状態を計算する
自発行動の自動送信	enabled=false / armed=false	候補の生成と、自動的な依頼送信は別である
記憶抽出	enabled=true、60秒間隔	会話から候補を作り、条件付きで自動採用する
Web検索	専用経路は未実装	検討資料の存在を実装済みと解釈しない

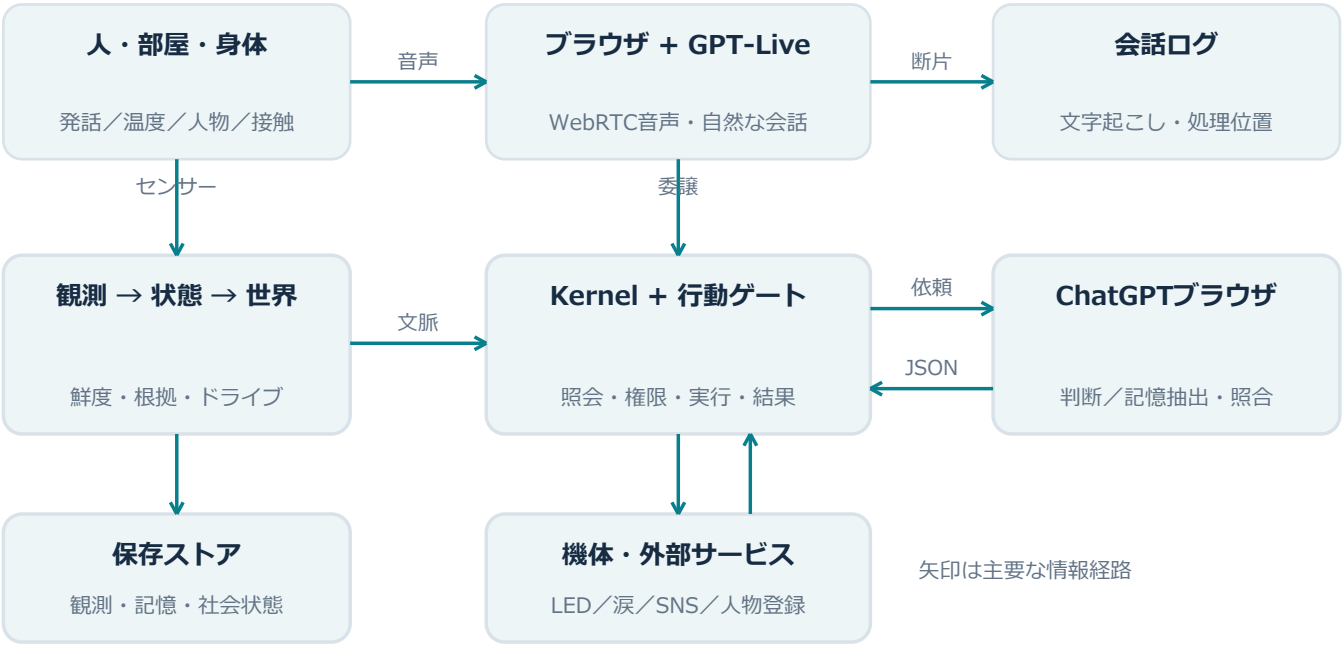
どこまで確認した資料か

コード・設定・プロンプト・既存テストを照合しています。Node.js 24.22.1で104テストが成功しました。ただし、テストの外部APIや機体は偽物に置き換えられています。実アカウントでの音声会話、物理的な音響エコー、機体の動作完了を本調査で証明したわけではありません。

> 「ファイルがある」「設定で有効」「実際に成功した」は、それぞれ別の状態です。以後の説明も、この区別に沿って読みます。

根拠（作業ツリー）：alice.js:60 / config.json / docs/live-web-search-options.md / test/live-runtime.test.js

02 全体像：会話と身体を結ぶ二つの経路



音声会話の主要経路は、**ブラウザのマイク → GPT-Live → ブラウザのスピーカー**です。カーネルはこの音声を毎回文字にしてChatGPTへ送り直す構成ではありません。Liveが会話を続ける横で、サーバーが別のWebSocketから文字起こしや委譲イベントを受け取ります。

もう一つは、**観測 → 状態 → 世界モデル → 判断用文脈 → ChatGPT → 監査 → 操作 → 結果**という身体側の経路です。実行してよい操作かを決めるのはカーネルです。Liveからの委譲だけでなく、人物や接触イベント、文字入力もこの経路を使います。

記憶抽出も同じChatGPTブラウザを使います。ただし目的は操作判断ではなく、会話から後で使える記録を作ることです。抽出した候補の照合を別の依頼で行い、コード側の条件を満たしたものを保存済み記憶として採用します。

なぜ分かれているのか

会話の自然なテンポと、身体操作の慎重さには異なる要求があります。LEDの制御先が遅くても会話は続けられます。一方、会話の勢いだけで機体を動かさないよう、操作の形式、権限、重複、実行結果を身体側で扱います。この分割がLive導入後の設計を理解する軸です。

根拠（作業ツリー）：alice.js:662 / src/live/kernel-bridge.js:56 / src/live/session-service.js:84

03 誰が何を決めるか：責任と権限

同じ「AIの発言」に見えるものでも、事実、解釈、操作提案では重みが異なります。役割を分けることで、生成された文章がそのまま物理世界の事実になることを防いでいます。

担当	持つ情報・権限	具体例
センサー・外部サービス	生の観測を送る	温度32℃、人物track 7を認識した
StateStore	最新の観測値と鮮度を持つ	32℃をいつ、どこから得たか
WorldModel	規則で世界の状態を導く	31℃以上なのでhot
DriveSystem	行動選択に使う必要度を計算	温熱快適性の必要度0.9
GPT-Live	聞く・話す・委譲を選ぶ	正確な身体確認が必要なら委譲する
ChatGPTバックエンド	文脈から判断案を返す	照会する、LED操作を提案する
監査・ActionGate	操作案を検証し許否を決める	今回の依頼に投稿の許可があるか
アダプター	実行先と通信する	HTTPでLEDコントローラーに送る
永続ストア	採用済み状態と履歴を持つ	記憶、社会的境界、操作結果

「提案」は確定状態ではない

自発行動の提案、長期記憶の候補、社会状態の候補は、それぞれ別のものです。行動提案は「こう振る舞う選択肢がある」という情報、記憶候補は「将来使う事実として残せそう」という情報です。社会状態候補は「この関係・約束・許可を保持すべき」という情報です。すべてを単一の「AIの記憶」にまとめると、コードの意図が見えにくくなります。

なお、Liveの通常発話は旧TTSのevaluateSpeechを経由しません。社会状態のsay禁止が存在しても、現在のコードだけからLiveの全音声を同じゲートで止められるとは読めません。権限の適用範囲は、通過する経路ごとに確認する必要があります。

根拠（作業ツリー）：src/behavior/index.js:26 / src/behavior/action-gate.js:79 / src/protocol/llm-response.js:117 / alice.js:1503

04 ファイル地図：どこを読むと何が分かるか

alice.jsは起動、サービスの結線、HTTP、ブラウザ監視、実行先との通信をまとめています。すべてのロジックがこのファイルにあるわけではなく、src以下の小さな部品を組み合わせています。

場所	役割	最初に追う入口
alice.js	全体を動かす配線	ingestObservation、buildCognitiveContext、executeGatedAction
src/behavior/	観測・状態・世界・必要度・提案	BehaviorArchitecture.observe
src/live/	Live接続、委譲、会話保存、記憶抽出	LiveSessionService、LiveKernelBridge
src/browser/	ChatGPT画面への入力と応答検出	sendBrowserJson、chatOutputMonitor
src/protocol/	AI応答の形式監査	LlmResponseProtocol.audit
src/memory/	記憶の候補・採否・検索・整理	MemoryStore
src/social/	関係・境界・約束	SocialStateStore
src/speech/	旧ASR、TTS、エコー対策、filler	AsrTranscriptService、TtsAdapter
dashboard/	画面、Live音声クライアント、診断	app.js、live-client.js
prompts / control_prompt.md	会話方針とバックエンドの出力契約	live-conversation.md
config.json / test/	現在の条件と回帰契約	数値と該当テストを照合する
archives/	過去の実験・設計	現在の動作の根拠にする前に現行コードと比較する

外にあるもの

人格本文は既定で隣のkokomi-personaリポジトリから読みます。機体側のLED・涙・センサー・人物登録サービスも、このリポジトリの外側です。このカーネル内にドライバーや画像認識モデルの全実装があるわけではありません。境界の外側について分かるのは、接続先、送る形式、期待する返答までです。

根拠（作業ツリー）：alice.js / package.json / config.json / src/behavior/index.js

05 起動時：過去を復元してから新しい入力を受ける

Node.jsがalice.jsを読み込むと、設定と環境変数を読み、各ストアやサービスを作ります。Liveが有効なら旧ASR・TTS・fillerの実効フラグがfalseになります。HTTPサーバー、センサーのポーリング、ChatGPTへの接続は、それぞれ非同期に準備されます。



runtimeReadyが守る順序

記憶、社会状態、観測ログの読み込みを待ってから、観測を順番に再適用します。再適用ではgenerateProposals=falseを指定し、昨日の「人が来た」を今日の挨拶候補として作り直しません。未完了のTTS、filler、操作意図があれば、再起動で中断・不明になった結果を追記します。

Liveの会話ログは別のconversationReadyで読みます。両方の準備が完了したあと、未処理の会話断片があれば記憶抽出を予約します。ただし、昨日のマイク接続や音声セッションを自動で復活させる仕組みではありません。

準備完了は一つではない

ChatGPTのChrome接続に失敗しても、APIとダッシュボードは診断用に残ります。逆にHTTPが応答しても、ブラウザへの依頼が成功するとは限りません。Liveのready表示も主にキーの存在を見ており、人格ファイルの読込、API権限、モデル利用可否まで事前に検証した表示ではありません。

根拠（作業ツリー）：alice.js:100 / alice.js:159 / alice.js:2048 / src/behavior/index.js:26

06 普通に話すとき：雑談はどう流れるか

「今日はちょっと疲れた」とマイクに話した場合を考えます。GPT-Liveが聞き、自然な会話を生成し、受信した音声をブラウザのAudio要素が再生します。Liveのプロンプトでは、雑談や会話中の情報だけで答えられる内容は委譲しない方針になっています。これはモデルへの指示であり、コードによる完全な分類器ではありません。

同時に起きていること

ブラウザはデータチャネルで文字起こしの断片を受け、画面へ表示します。サーバーもsidebandから同じ種類の断片を受けて、ConversationStoreへ保存します。役割はuserかassistant、時刻、セッションID、根拠IDを持つ記録になります。

ユーザーの文字起こしを受けると、カーネルはinteraction.live_activityを取り込みます。ここでは「最近やり取りがあった」という時刻を更新します。文字起こしの断片ごとにChatGPTの認知ターンを作る処理ではありません。社会的接触の必要度が、会話中なのに増え続けることを防ぐ意味があります。

その場の会話と長期記憶は別

会話に出た内容が、直ちに採用済み長期記憶へ入るわけではありません。会話ログへの保存、抽出対象の選択、記憶候補の生成、照合、コード側の採否を経ます。「Liveがその場で話を覚えている」ことと「次回もカーネルから取り出せる記憶になった」ことは異なります。

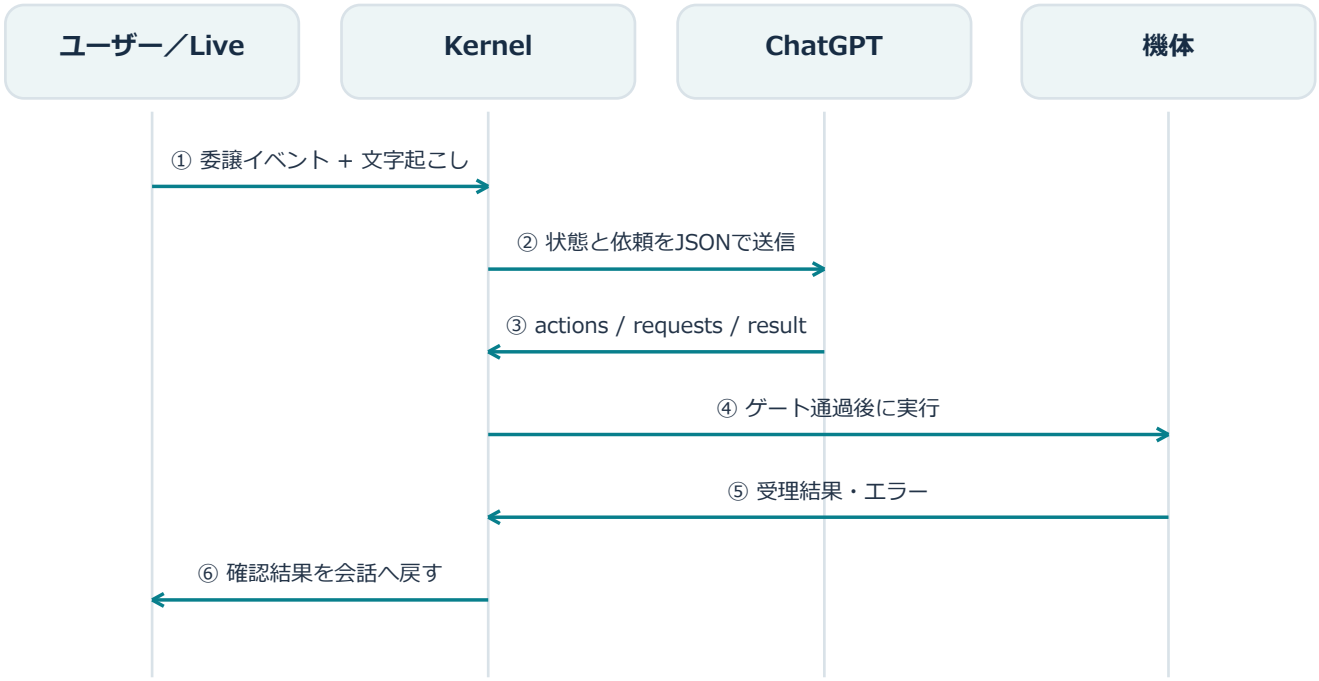
> 日常会話が成り立つために、毎回カメラや機体やChatGPTブラウザの応答を待つ必要はありません。ただし必要な操作判断や記憶処理では、別途バックエンドの応答が必要です。

ブラウザ側は音声認識のエコー抑制、ノイズ抑制、自動音量調整を要求しています。これらの指定が、実環境でどれだけ効くかは別途確認が必要です。旧音声経路の半二重制御がLiveにそのまま適用される構成ではありません。

根拠（作業ツリー）：prompts/live-conversation.md / dashboard/live-client.js:23 / src/live/session-service.js:193 / alice.js:680

07 身体に関わる依頼：Liveから何を委譲するか

「今の部屋は何度？」「目の前に何が見える？」「LEDを青にして」のように、会話だけでは確かめられない事柄は、Liveのプロンプトでバックエンドへの委譲対象になっています。「覚えて」、処理中の依頼の訂正・取り消しも対象です。



委譲イベントだけでは依頼文がない

session.delegation.createdには委譲IDなどのメタデータが来ます。コードは、このイベントに架空の依頼文を付け足しません。保存された会話から、委譲のoffset_ms以前に始まった最新のユーザー発言を探し、最後の断片が届いてから1,200ms落ち着くのを待ちます。待つ上限は15秒です。

依頼が確定できたら、interaction.user_inputとして観測を作り、状態・世界・記憶・社会状態を付けた文脈を作ります。直近12グループの会話も追加します。依頼文を十分に確定できなければ、操作内容を聞き直すための結果をLiveへ返します。

ChatGPTはlive_controlという専用契約で、result、actions、requestsを返します。操作はゲートを通り、照会があれば次の往復で結果を渡します。通常は最大4往復です。確認結果はcommentaryとしてLiveへ送り、会話の中で説明させます。

根拠（作業ツリー）：src/live/session-service.js:225 / src/live/kernel-bridge.js:13 / src/live/kernel-bridge.js:56 / config.json

08 待ち行列と取り消し：遅い処理をどう扱うか

ChatGPTの入力欄は一つです。操作判断と記憶抽出を無秩序に送り込むと、別の依頼の応答を取り違えます。BackendQueueは**送信から応答までの一往復全体**を直列化します。さらに入力・画像添付などのブラウザ操作には、alice.js側の操作キューがあります。

仕組み	現在の動き	必要な理由
BackendQueue	一件ずつ最後まで実行	記憶抽出の応答を操作判断と混ぜない
優先度	操作100、記憶0	待機中の記憶より操作確認を先にする
実行中の仕事	途中で強制割込みしない	画面上の一往復を壊さない
task.revision	会話の依頼世代と比較	訂正前の判断を実行しない
委譲TTL	作成から5分	古い依頼を際限なく生かさない
forAction確認	activeかつ未終了を要求	会話が終わったあとの未実行操作を防ぐ

「やめて」が届いたら

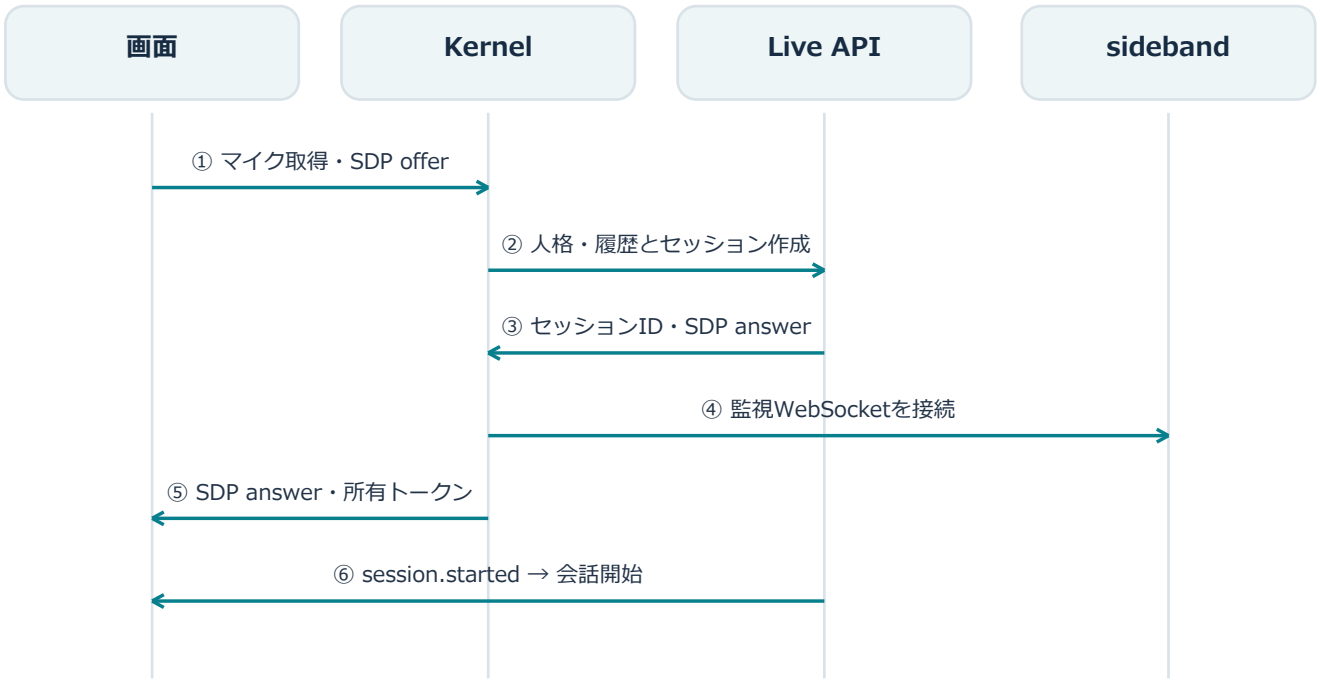
ユーザー文字起こしの最新グループに、やめて、取り消、キャンセル、中止、やっぱり、じゃなく、ではなく、cancel、insteadなどが含まれると、セッションのrevisionが増えます。以前のrevisionで始まった委譲はisCurrentに通らなくなります。処理の開始前、応答後、操作直前などで繰り返し確認します。

この処理は、すでに機体へ送ってしまった命令を巻き戻す装置ではありません。すべての自然言語の訂正を理解する仕組みでもなく、特定語と世代番号による保守的な無効化です。実行中のHTTP通信やブラウザモデルの生成自体を中断する処理と、古い結果を捨てる処理を区別して読んでください。

> 操作が優先されても、実行中の記憶抽出が終わるまでは待つ場合があります。その間もLiveの音声会話は別経路で続けられます。

根拠（作業ツリー）：src/live/backend-queue.js / src/live/kernel-bridge.js:35 / src/live/session-service.js:234 / src/live/session-service.js:221

09 会話開始：マイクからセッション確立まで



開始ボタンでまずマイクを取得し、RTCPeerConnectionに音声トラックとoai-eventsデータチャネルを追加します。SDPは接続条件を交換するための記述、ICEは接続に使える経路を集める手続きです。ICE待ちは既定10秒です。

サーバーは人格本文と会話方針を読んでから、有料セッションの作成を呼びます。人格が見つからない場合は、その呼出しの前に失敗します。現在の身体由来の話し方も初期instructionsへ含めます。モデルはgpt-live-1、声はharemaという設定です。

二つの接続がそろう

Live APIからセッションIDとSDP answerを受け、サーバーはattach用WebSocketを接続します。その後、ブラウザへSDP answerとランダムな所有トークンを返します。sidebandのopenだけでは音声開始と見なしません。session.started、または所有ブラウザがstartedを受けたというheartbeatでactiveへ進めます。

ブラウザへAPIキーは渡しません。サーバーは一度に一つのセッションを所有します。新規作成中や、前の終了を確認できない状態での二重開始は409で拒否します。開始処理の途中で終了が押された場合も、後から返ったセッションを閉じる分岐があります。

根拠（作業ツリー）：dashboard/live-client.js:23 / src/live/session-service.js:48 / src/live/session-service.js:209 / alice.js:669

10 会話終了：マイク停止と終了確認は別の段階

終了ボタンを押すと、ブラウザは先にマイクのトラック、Audioの再生、データチャネル、PeerConnectionを解放します。その後、サーバーへ所有トークン付きでstopを送ります。利用者の手元の音声停止を、ネットワークの返答待ちで遅らせない順序です。

状態・条件	意味	次に起きること
idle	停止・終了確認済みなど	開始操作を受けられる
connecting	作成・接続中	session.startedを待つ
active	音声開始を確認した	文脈同期や委譲を行う
closing	終了要求を送った	session.closedを待つ
error	何らかの失敗	lastErrorとcloseConfirmedを併せて読む

終了を確認する理由

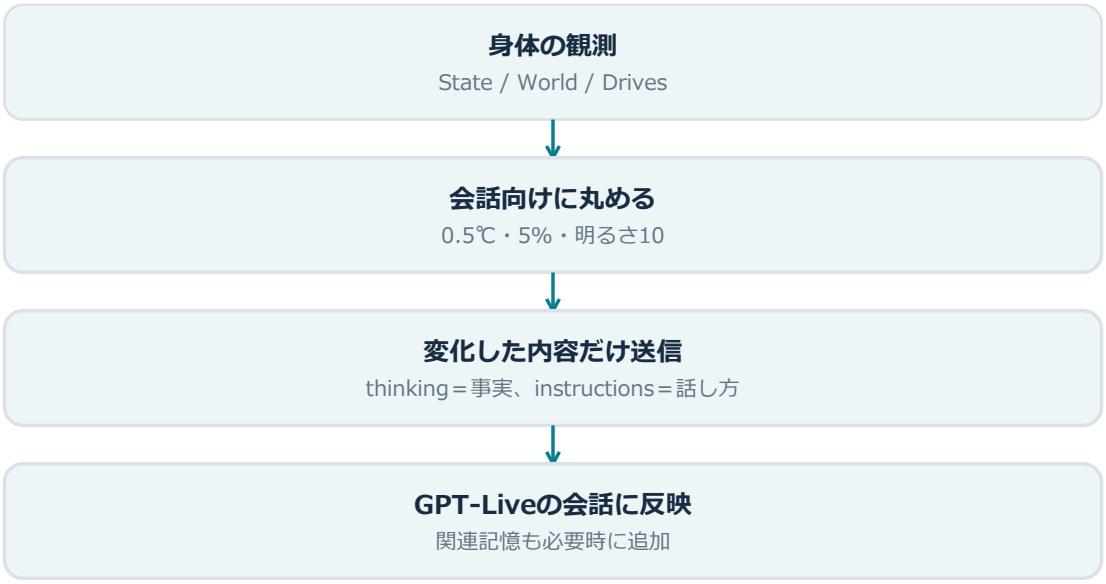
サーバーはsession.closeを送り、最大5秒session.closedを待ちます。確認できると最終usageを会話ログのkernel記録に保存します。閉じる途中に受け取った文字起こしも、可能な範囲で保存します。会話終了時には残った記憶抽出も予約します。

確認できなければcloseConfirmedはfalseのままで、新しい有料セッションを作りません。ブラウザ側も所有トークンを残し、終了再試行に使います。「こちらのマイクは止まった」ことと「相手側セッションが確実に終了した」ことを分けるためです。

ブラウザは5秒ごとにheartbeatを送り、サーバーは20秒以上途絶えると終了を試みます。ページを離れるとsendBeaconも使いますが、OSやブラウザの強制終了時に必ず届く保証はありません。サーバーのSIGINT/SIGTERMでも終了を試みますが、非同期の記憶抽出全体の完了まで保証する終了手順ではありません。

根拠（作業ツリー）：dashboard/live-client.js:151 / dashboard/live-client.js:172 / src/live/session-service.js:329 / src/live/session-service.js:361 / alice.js:1963

11 Liveへ渡す身体状態：全部を毎回送らない



身体側は1秒ごとに温湿度や明るさを更新しますが、Live向けの保守・同期は既定2秒周期です。bodyContextは、会話に必要な短い説明へ変換します。室温は0.5℃刻み、湿度は5%刻み、明るさは10刻みで丸めます。在室、温熱状態、撫でられているか、見えている人物（最大5人）も含みます。

事実・話し方・結果を使い分ける

thinking.appendは背景情報、instructions.appendは話し方の変更、commentary.appendは確認結果の返却に使います。事実の文字列が前回と異なる場合だけ送るため、小さな測定ゆらぎで毎回文脈を増やしません。正確な値が必要なら、バックエンドから鮮度付きの状態を照会します。

intoxicationという外部信号は、0.2未満、0.2以上0.6未満、0.6以上という帯に分けて話し方の指示に変えます。未知や期限切れなら酔っているとも素面とも断定しません。これは声の演技への指示で、音響パラメーターを決定的に制御する機構ではありません。

初期の話し方は開始時のinstructionsへ入れ、同じ指示を開始直後に再送しません。会話中の指示追加は発話を中断し得るためです。身体の数値と話し方は別に比較され、事実だけ変わったときに話し方の指示まで毎回追加する構成ではありません。

根拠（作業ツリー）：src/live/body-context.js / src/live/session-service.js:270 / src/live/session-service.js:58 / docs/live-debug-2026-10-29.md

12 文脈の予算：AIへ何をどれだけ渡すか

コンテキストとは、モデルがその時点の返答を考えるために参照する情報です。会話履歴、人格設定、身体状態、過去の記憶などが入ります。無制限に追加すればよいわけではなく、重要な事実を選び、長さを抑えます。

入力	現在の上限・目安	選び方
Live初期情報	6,000という設定予算	要約・身体・関連記憶におよそ半分、残りに最近の会話
初期履歴	最大24グループ	新しいものを優先し、kernelの記録は除く
追加情報1回	450という設定予算	Unicode文字の途中を切らずに分割
Liveの関連記憶	最大5件	最近4グループ中のuser発言を検索語にする
バックエンド依頼本文	8,000という設定予算	長すぎる場合はfitTextで切る
バックエンド会話	最大12グループ	1グループの予算も制限する
保存用の要約	1,600という設定予算	長期記憶本文とは別に保持する

名前にTokensとあっても実装はUTF-8バイト

textCostはトークナイザーを呼ばず、UTF-8バイト数を数えています。日本語1文字は通常複数バイトなので、450は450文字ではありません。このコードは実トークン数を正確に測るのでなく、余裕を持って小さく収める方針です。総依頼の全項目に一括で8,000を割り当てる厳密な総量計算でもありません。

追加送信には最大32件の未確認イベント枠があります。確認が滞れば新しい文脈を保留し、警告を出します。操作結果にはpendingResultsという別の待機列もあり、同期処理で再送を試みます。音声会話そのものを止める条件とは分かれています。

なぜ記憶は必要なときだけか

すべての過去を毎回入力すると、長くなるだけでなく、今の依頼と関係のない話が混ざります。初期検索で候補がないときだけ、採用済み長期記憶の末尾から少数を補います。会話中は同じ記憶IDを二度送らないよう追跡します。

根拠（作業ツリー）：src/live/text-budget.js / src/live/session-service.js:61 / src/live/session-service.js:246 / src/live/session-service.js:296 / config.json

13 Liveの警告：拒否・無音・音声欠落を分ける

イベント拒否は、必ずしも音声回線の断線ではありません。LiveServiceはerrorイベントを受けると、種類・コード・理由・対象client_event_idを記録します。対象の追加送信を未確認集合から外し、音声とイベントの受信は続けます。秘密らしいキー文字列は記録前に伏せます。

無音の警告

LiveAudioMonitorは、マイク側と受信音声側の音量を既定1秒ごとに読みます。両方がしきい値0.02未満のまま60秒経つと、画面通知とローカルの二連ビープを出します。無音が続けば60秒ごとに再通知し、音量の活動や文字起こしがあれば時間をリセットします。無音だけで自動切断はしません。

警告音はWeb Audioで作る880Hzの短い音です。LiveやChatGPTへ「警告をしゃべって」と依頼しません。ブラウザが音声再生を許可していなければ、再生許可ボタンを案内します。

診断項目	見ているもの	分かる範囲
lostDelta / discardedDelta	損失・破棄の増分	その区間に受信上の問題があったか
concealedDelta	補間された音声サンプル	欠落などを再生側が補ったか
jitterMs / bufferMs	到着ゆらぎ・待機遅延	通信と再生待ちの傾向
playback	paused / muted / readyState	ブラウザの再生状態

文字起こしは音声再生の証明ではない

「のんびり」の一部が聞こえないような現象は、文字起こしに全文があっても否定できません。生成・割込み・伝送・OSやブラウザの再生を切り分ける必要があります。現在の診断は直近180サンプルをメモリー内に保持し、音声自体は録音しません。特定の音節が欠けた原因を、この統計だけで断定することもできません。

根拠（作業ツリー）：src/live/session-service.js:195 / dashboard/live-audio-monitor.js:56 / dashboard/live-audio-monitor.js:89 / docs/live-debug-2026-10-29.md

14 観測：外から来た情報を共通の封筒に入れる

観測とは「どこかの入力元が、ある時刻にこう報告した」という記録です。温度、人物、接触、操作結果のように内容が違っても、共通の外枠にそろえます。createObservationがその境界を作ります。

フィールド	平易な意味	例
id	この観測だけを指す名前	obs_にUUIDを付ける、または外部の安全なID
type	何の種類の出来事か	sensor.environment_sample
source	報告元	bme280、touch、live_delegation
observed_at	実際に観測したとされる時刻	センサー側のタイムスタンプ
received_at	カーネルが受け取った時刻	遅延や鮮度の区別に使う
confidence	入力側が付けた確信度	0〜1、既定1
payload	中身	temperature、text、track_idなど

IDは最大128文字の安全な文字、typeはドットで区切る小文字識別子、sourceは非空・最大128文字です。payloadがオブジェクトか、時刻が解釈できるか、confidenceが有限な0〜1かを確認します。外枠とpayloadの浅いコピーをfreezeしますが、深い入れ子まで完全不変にする処理ではありません。

外枠だけでは内容の正しさは保証しない

人物イベントのidentity、接触のsensor_id、内部信号の値などはHTTP側で別に検証します。StateStoreも温度が有限の数かなどを確認します。観測を一種類の関数に通すからといって、すべてのpayloadを共通の厳密スキーマで保証しているわけではありません。

外部が「こう報告した」と記録することと、本当にそうだったと実世界で証明することも別です。観測IDとsourceを残すのは、後から「どの報告が判断の根拠か」をたどれるようにするためです。

根拠（作業ツリー）：src/behavior/observation.js:21 / alice.js:164 / alice.js:1343 / src/behavior/state-store.js:63

15 状態：観測履歴から「いま」を取り出す

StateStoreは、入力された観測を種類ごとに適用する処理です。温度なら最新値を差し替え、人物ならtrackの集合を更新し、接触なら触れている部位を増減します。毎回revisionを増やし、snapshotで読める形にします。

状態の領域	主な内容	主な更新元
environment	温度、湿度、気圧、明るさ	センサーのサンプル
perception	人の存在、見えている人、最近の音	vision、audio.classification
interaction	接触、最後の入力、旧ASRの途中結果	touch、interaction、asr
output	旧TTSとfillerの実行状態	speech.output_*、speech.filler_*
action	最後の意図と結果	action.intention、action.outcome
provenance	最後の観測ID・種類	適用したすべての観測

値だけでなく、単位、観測時刻、報告元、確信度、根拠の観測IDを持ちます。たとえば「室温28℃」だけを保持するより、「10秒前にBME280から来た28℃」のほうが、その数字を今の判断に使ってよいか決めやすくなります。

snapshotは現在時刻で鮮度を計算する

新しい観測がなくても、時計が進めば古くなります。snapshotを作るたびに有効期限を見てknownかstaleかを変えます。このため、revisionが同じでも時間経過によって公開されるstatusは変わることがあります。revisionは「世界に変化がなかった」ことの証明ではなく、観測適用の版数です。

状態と履歴の違い

StateStore自体はメモリー内の最新状態で、全履歴のデータベースではありません。再起動後の状態はEventStoreの観測ログから作り直します。一方、長期記憶や社会的境界は別ファイルです。「状態を永続化する」は、ここでは観測の再適用で復元できるという意味を含みます。

根拠（作業ツリー）：src/behavior/state-store.js:63 / src/behavior/state-store.js:150 / src/behavior/event-store.js:28 / alice.js:100

16 鮮度：unknown・stale・falseを区別する

「人がいない」と「人がいるか分からない」は、同じではありません。確かめられていない状況をfalseとして扱うと、判断が過剰に確定します。カーネルは、値にstatusを付けてこの違いを表します。

表現	意味	温度の例
known	現在の規則では使える根拠がある	5秒前に得た28℃
unknown	まだ使える根拠がない	一度も温度を取得していない
stale	過去の値はあるが期限を過ぎた	最後の28℃が20秒前
knownかつfalse	明示的な否定の状態	人物消失でtrackがなくなった、など

現在の設定の鮮度は、温湿度・気圧15秒、明るさ15秒、音5秒、人物5分、接触30秒です。StateStore単体の人物既定値は10秒ですが、alice.jsから渡すconfig.jsonでは5分に上書きされます。クラスの既定値より、実際の呼出しの設定を優先して読む必要があります。

例：32℃のあと通信が止まった

最初の15秒以内は温度32℃がknownで、世界モデルはhotです。期限を過ぎると、状態には32という過去値が残ってもstatusはstaleになります。WorldModelはknownな温度だけを使うので、温熱状態はunknownへ変わります。現在も暑いと断定せず、「最新の測定がない」と扱うためです。

人物の場合、期限切れtrackが残っているだけなら在室はstaleになります。明示的なperson_disappearedで全trackが消えるとknownのfalseになります。接触も、開始だけ届いて終了が来なければ、30秒後にはstaleになります。

> unknownは「問題がない」でも「危険」でもありません。判断に使える情報が足りないという意味です。必要なら確認する、という次の行動につながります。

根拠（作業ツリー）：src/behavior/state-store.js:19 / src/behavior/state-store.js:150 / src/behavior/world-model.js:9 / config.json

17 世界モデル：測定値を意味のある事実へ変える

世界モデルとは、このプロジェクトでは「今の部屋や活動を、扱いやすい事実に整理したもの」です。大きな3D地図や、未来の動きを学習するニューラルモデルを指しているわけではありません。WorldModelはStateStoreのsnapshotから決まった規則で値を計算します。

派生する事実	現在の規則	元の情報
room.is_occupied	人の存在がknownならその値、ほかはunknown	person_present
thermal_condition	18℃未満cold、27℃未満comfortable、31℃未満warm、31℃以上hot	knownな温度
lighting_condition	40未満dark、75未満normal、75以上bright	knownな明るさ
is_quiet	quiet / lowならtrue、それ以外の既知levelならfalse	音分類のlevel
someone_is_talking	labelがspeechならtrue	knownな音分類
character_is_being_petted	接触がknownならその値	being_petted
last_action_status	最後の操作結果のstatus	action.last_outcome

各派生事実にはbased_onという根拠の名前やconfidenceが付きます。未知の場合はno_fresh_person_evidenceなどの理由を返します。どのstate revisionから作ったかも保持するので、説明と測定値の対応を調べられます。

なぜLLMに毎回「暑いか」を決めさせないのか

温度が同じなのに、会話の流れだけで「暑い」「寒い」が変わると、実験の条件が揃いません。規則をコードに置けば、閾値を確認し、同じ入力で同じ派生結果を再現できます。そのうえでLLMが、どんな言葉でどう応じるかを選べます。

ただし、この閾値は人間の体感を正確に再現したものではありません。湿度を含む暑さ指数の計算もしていません。brightnessはraw値であり、照度のluxへ校正されていません。「hot」はこの設定のラベルだと理解してください。

根拠（作業ツリー）：src/behavior/world-model.js / config.json:76 / control_prompt.md:365

18 人物と視覚：見えている人と画像理解は別経路

人物イベントは、外部のDeepSort系サービスから/yolo_eventへ送られます。カーネルはtrack_id、identity、時刻、メッセージなどを確認し、vision.person_recognized、person_unknown、person_enrolled、person_disappearedの観測へ変換します。

種類	分かること	状態への反映
person_recognized	登録済み人物と照合できた	trackに名前・person_id・位置を保持
person_unknown	人物はいるが誰か不明	存在は保持し、名前を推測しない
person_enrolled	登録側が完了を報告した	認識済みidentityとして更新
person_disappeared	そのtrackが見えなくなった	trackを削除

recognizedでは、nameとperson_idがあり、distanceがthreshold以下であることを検査します。distanceは顔特徴などの照合距離で、ここで検査するのは外部の返答の整合です。カーネルが画像から顔埋め込みを計算しているわけではありません。同じevent_idの重複はプロセス内で最大1,000件の範囲で抑止します。

名前が分かることと、話し手が分かること

trackは画面内の追跡対象の番号です。名前との対応や、マイクで話す人との対応が常に取れるとは限りません。複数人が見える場合の人物登録は、誰を指すか確認するようプロンプトに指示があります。行動ゲートも現在の文脈にtrackがあるか、今回の発言に名前が含まれるかを確認します。

「何が見える？」は別の処理

場面説明の依頼では、カメラのsnapshotを取り、ChatGPTの同じ依頼に画像を添付します。人物イベントの名前情報だけで画像全体を理解したことにはしません。旧LLaVA関数は残っていますが、現行の視覚照会からは呼ばれません。

根拠（作業ツリー）：alice.js:1311 / alice.js:1343 / src/behavior/state-store.js:78 / alice.js:1015 / alice.js:2012

19 接触と音分類：会話以外の入力も意味へ変える

接触は/touch_sensor_inputで受けます。sourceはtouch、種類はtouch_startedかtouch_endedです。sensor_idを設定の表で身体部位へ変換し、petting_startedかpetting_endedという意味付きの観測にします。

入力	現在の対応	状態・派生結果
touch_01	head	頭を撫でられている
touch_02	hand	手に触れられている
touch_03	shoulder	肩に触れられている
音label=speech	音分類の文字列	someone_is_talking=true
音level=quiet / low	大まかな音量区分	is_quiet=true
音level=high / loud	大まかな音量区分	sensory_rest=0.8

StateStoreは複数部位の接触をMapで持ちます。headの開始のあとhandの開始が来れば、両方が有効な間は両部位が一覧に出ます。headの終了だけ届けばhandは残ります。開始時刻から30秒を超えた部位は、開始のままでも現在の接触として使わなくなります。

音分類と音声認識の違い

/audio_classificationは「話し声」「犬の声」などのlabelと音量区分を受けます。話した文章を文字にするASRとは役割が異なります。現在のforwardToLlm=falseでは、音分類のたびにChatGPTへ依頼を送らず、状態や世界モデル、ドライブを更新します。

人物と接触イベントは現在のHTTP実装ではsendObservationToChatGPTを呼びます。Live有効時もバックエンドの依頼が作られ得ます。自発行動の自動tickが無効だからといって、センサー由来の認知処理がすべて停止しているわけではありません。

> 接触の意味付けには実装上の約束があります。すべてのtouchをpettingと呼ぶ現在の変換は、触れ方の強さや快・不快まで識別しているという意味ではありません。

根拠（作業ツリー）：alice.js:1790 / alice.js:1835 / src/behavior/state-store.js:82 / src/behavior/world-model.js / config.json

20 ドライブ：行動を考えるための必要度

ドライブは、会話や行動の優先度を考えるための0〜1の指標です。ここでは生物学的な感情や痛みの証明ではありません。「室温が高いので温熱調整を考える必要度が高い」のように、機能上の要求を数値にしたものです。

指標	現在の計算	解釈
thermal_comfort	hot=0.9、cold=0.7、warm=0.35、comfortable=0	温熱快適性へ注意を向ける度合い
social_contact	最後の有意味な交流からの経過時間 / 4時間	人と関わる必要度の目安
在室による補正	在室trueならsocialを0.35倍	人が近くにいる状況を反映
接触による補正	撫でられているtrueならsocial=0	接触中の要求を抑える
sensory_rest	high / loud=0.8、medium=0.35、ほか既知は原則0	感覚的な休息の必要度
external	/internal_stateから有限の0〜1を受信	外側で計測・計算した身体信号

有意味な交流として、文字入力、Liveのユーザー活動、接触開始、人物認識・登録を扱います。初期に交流時刻がなければsocialは0.25です。外部信号は5分を超えると一覧から落とします。0.6以上の数値を強い必要度として並べ、上位3件をdominantにします。

身体内部の信号をどう入れるか

/internal_stateはsignalsという名前と値の組を受けます。1〜32件、名前は小文字識別子、値は0〜1です。intoxicationもこの一般的な入口を使います。エタノール量から酔い度を校正して計算する部分は、現在のカーネルにはありません。

この層は、必要度が高いから直接モーターを回す処理ではありません。必要度を文脈に載せたり、自発行動候補の理由に使ったりします。欲求の表現と実行許可を分けることで、会話の演出と身体の制御を混同しにくくしています。

根拠（作業ツリー）：src/behavior/drive-system.js / alice.js:1864 / src/live/body-context.js / config.json:98

21 自発行動：候補を出すことと動くこと

SpontaneousBehaviorEngineは、観測と世界モデル、ドライブから「今ならこの振る舞いを考えてもよい」という候補を作ります。候補にはID、優先度、理由、関連情報、期限、クールダウンキーが付き、executionはcognitive_review_requiredです。直接の機体命令ではありません。

候補	発生する主な条件	優先度 / 間隔
social.greet_arrival	人物の認識・登録・未知人物イベント	80 / 2分
social.respond_to_touch	接触開始	85 / 10秒
environment.mention_temperature	人がいてcoldかhot	cold 55、hot 62 / 15分
environment.consider_lighting	人がいてdark	50 / 15分
homeostasis.consider_regulation	最大ドライブが0.75以上	50 + level×35を丸める / 5分

候補の有効期間は30秒です。クールダウンの時刻は候補を作った時点で更新され、実際に発話・操作した時刻ではありません。挨拶も「新しい人物が来た」と厳密に差分検出するのでなく、該当する人物イベントとクールダウンで候補になります。

現在の有効範囲

定期的な自発tickはenabledとarmedの両方がtrueの場合だけ動きます。現在は両方falseです。候補生成は通常の観測で続き、別の依頼の文脈に載ることはあります。手動の/api/behavior/tickは別入口として残り、同じenabled/armed確認を通していません。

自発tickが動いても最低優先度は70です。低い温度・暗さ候補は単独では対象になりません。自発ターンで許可される外部操作一覧も現在は空です。旧発話ゲートには在室確認もありますが、Liveの自然発話全体にそのゲートが掛かるわけではありません。

候補の貸出し

文脈を送るときは候補を一時的にleaseとして確保します。送信に失敗すれば期限内のものを戻し、成功すれば確定します。これにより、送れなかっただけで候補が消えることを減らします。

根拠（作業ツリー）：src/behavior/spontaneous-behavior.js / src/behavior/index.js:70 / alice.js:1892 / alice.js:1971

22 認知文脈：ChatGPTへ渡す一回分の資料

認知という言葉は、ここでは状況を受け取り、意味を考え、返答や行動を選ぶ処理を指します。ContextComposerは、その判断に使う資料をcognitive_contextというJSONへまとめます。内部の全データをそのまま送るのではなく、必要な形へ投影します。

項目	入るもの	なぜ必要か
turn_id / protocol_version	依頼の識別子と形式の版 1.2	どの依頼への返答か確かめる
trigger	今回のきっかけの観測	何に反応しているかを明確にする
persona	人格IDと短縮した版	人格本文と状態の責任を分ける
state	温度・在室・接触などの簡潔な値	現状の判断材料
world	温熱・明るさなどの派生事実	数値を意味へつなぐ
memory	検索で得た採用済み記憶	関連する過去を参照する
social	関係・境界・約束・見えている人	人間関係と許可の文脈
drives	必要度と身体信号	身体由来の優先事項
last_action_outcome	最近の操作結果	成功を推測しないため
behavior_proposals	任意の行動候補	自発的な応答の選択肢

何が省かれるのか

内部状態の細かい時刻や報告元を毎回全部送りません。knownの数値は丸め、未取得や期限切れはunknown、staleという文字列にします。記憶や提案がなければ、そのブロックを省きます。ただし記憶の根拠IDや候補のpriorityなど、判断に必要な情報は残します。

TurnRegistryは最大32件の待機ターンを保持し、5分で期限切れにします。正しい応答を一度受け取ればconsumeし、同じ返答を繰り返し実行しません。長期保存とは異なり、待機ターンはプロセス内の対応表です。

根拠（作業ツリー）：src/behavior/context-composer.js / alice.js:173 / src/behavior/turn-registry.js

23 ChatGPTブラウザ：API呼出しではない判断経路

身体判断と記憶抽出のバックエンドは、ChromeのChatGPT画面です。Puppeteerが既定の127.0.0.1:9222へ接続し、URLにchatgpt.comなどを含む最初のページを選びます。見つからなければ新しいページを作ります。モデル名をAPIの引数で固定する構成ではないため、使われるモデルは実際のブラウザ会話側に依存します。

入力するまでの手順

sendBrowserJsonはJSON全体を一括挿入します。入力欄に文字が残っていれば選択・削除し、挿入後に内容がJSONと完全一致するか確認します。利用可能で表示されている送信ボタンを探し、DOMのclickを呼び、入力欄が空になることも確認します。文字ごとの長いタイピングを避け、日本語や絵文字や改行の欠落を検出するためです。

BrowserBackendは最初の依頼にcontrol_prompt.mdの初期説明を同梱します。説明だけで一回応答を発生させず、最初の仕事と一つのturn_idで対応させます。入力やキュー待ちを除き、送信後から120秒の応答待ちを始めます。

出力をどう読むか

chatOutputMonitorは300msごとにassistant側のDOMを探します。新しい属性と旧属性に対応し、ユーザー自身が貼ったJSONは除外します。開始時に見えていた履歴を境界として、新しい変更をonPartialOutputへ届けます。カーネル側はJSONが閉じたかを波括弧・文字列・エスケープを見ながら抽出し、さらにturn_idと形式を監査します。

> DOMを読むので、ChatGPT画面の変更は実行時の依存です。APIが正常でも「入力欄が見つからない」「送信済みだが応答を拾えない」という故障があり得ます。

応答の生成終了を専用APIで受けるのでなく、DOMから完全なJSONが得られた時点で監査に進みます。pending turnがない過去のJSONは実行しません。画像添付には別の待機・一時ファイル処理があり、その経路ではページを前面へ出す処理も残っています。

根拠（作業ツリー）：alice.js:2048 / src/browser/chat-input.js / src/browser/output-monitor.js / src/live/browser-backend.js / alice.js:853

24 形式監査：AIのJSONをどこで止めるか

LlmResponseProtocolは、返ってきた内容がこのカーネルの契約に合うかを確かめます。文章としてもっともらしいかを評価する処理ではなく、形式、値の範囲、許可された項目、タスクごとの出力形を検証します。

契約	許される中心の出力	意図
旧通常ターン	speech、emotion、intensity、actions、requests、候補類	旧来の一括した会話・操作
live_control	turn_id、result、actions、requestsのみ	Liveが話し、バックエンドが事実と操作案を返す
live_memory	turn_idとmemory_extractionのみ	抽出中に操作を実行させない
live_memory_review	turn_idとmemory_verificationのみ	照合だけに役割を絞る

通常契約ではspeechは最大2,000文字で、存在するときだけemotionと0〜1のintensityが必要です。感情はneutral、happy、calm、sad、angry、surprised、fear、thinkingの8種類です。actionsは最大1件、requestsは最大5件です。live_controlのresultも1〜2,000文字で、speechや記憶・社会候補は許されません。

対応と形式は別々に検査する

先にTurnRegistryからpendingの文脈を得ます。存在しなければ、形式が正しくても処理しません。その文脈のtaskを使って監査し、一般の記憶・社会候補では根拠イベントがストアに存在することも確認します。成功してからターンを消費し、Liveの待機中ジョブへ応答を返すか、旧実行経路へ渡します。

監査の限界

形式が正しいresultにも、事実誤認はあり得ます。根拠IDが実在するだけでは、候補の文章の意味まで証明できません。そのためLive記憶には原文照合と別依頼の確認があり、機体操作には実行先の結果が優先されます。形式監査、意味の確認、実行結果は異なる層です。

根拠（作業ツリー）：src/protocol/llm-response.js:117 / src/protocol/llm-response.js:25 / alice.js:2115 / alice.js:927

25 行動ゲート：提案を実行してよいか判断する

形式監査を通ったactionでも、実行が許されるとは限りません。ActionGateは、同じターンの実行数、重複、短時間の連続操作、採用済みの禁止、今回の許可などを順に確認します。

確認	現在の条件	防ぐこと
ターンと文脈	両方がある	根拠のない操作
1ターンの数	最大1操作	一度の判断で複数の副作用
同じ内容	30秒は抑止	再送や生成の反復
操作別間隔	LED 2秒、涙10秒、投稿5分、人物登録1分	高頻度の操作
社会的境界	acceptedのdenyなら拒否	保存済みの禁止との矛盾
投稿・人物登録	現在のuser_inputによる明示許可	過去の許可や自発判断による実行
自発ターン	許可一覧にある種類だけ	未装備の自律的な外部操作

重複はactionのキー順を整えたJSONのSHA-256で比較します。クールダウンは操作種類単位で、LEDなら色が違っても間隔制限の対象です。許可された段階で実行数・時刻を更新するため、あとで通信に失敗しても直後の再実行が通るとは限りません。

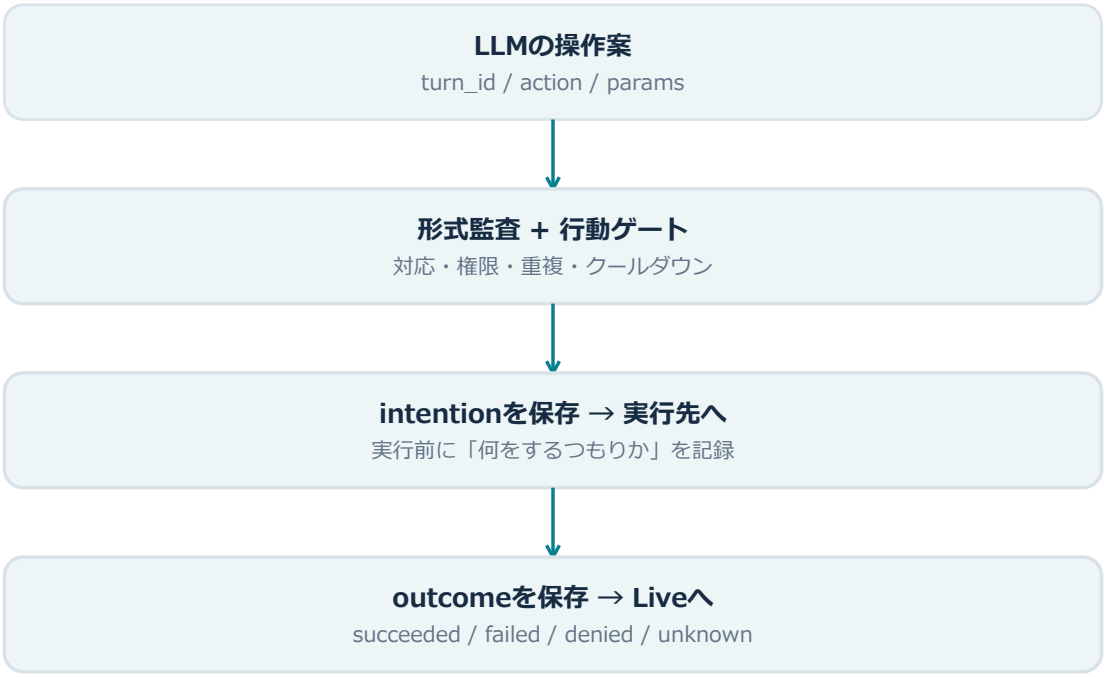
許可は自然言語の全理解ではない

投稿と人物登録の許可は正規表現で判定します。否定のパターンを先に拒否し、肯定的な依頼の形を探します。すべての日本語の曖昧さを解決するものではありません。人物登録では文脈内のvisible_peopleと名前の文字列も確認しますが、その情報の基準は元の依頼文脈です。

Liveで照会を重ねても、操作の権限と1操作制限にはrootのturn_idと文脈を使います。新しい照会ターンを作っても、一つの依頼で操作数を増やせないようにしています。

根拠（作業ツリー）：src/behavior/action-gate.js:60 / src/behavior/action-gate.js:24 / src/live/kernel-bridge.js:69 / config.json:177

26 フィードバック：命令したことと起きたこと



フィードバックは、行動のあとで得た結果を次の判断へ戻すことです。「青にしよう」と決めたことを成功と見なすと、通信が失敗しても青になった前提で話してしまいます。ここではintention（実行意図）とoutcome（実行結果）を分けます。

許可された操作は、先にaction.intentionを観測として保存し、そのあと実行先へ送ります。返答が成功ならsucceeded、例外ならfailedです。ゲートで止めればdenied、再起動で未完了の意図だけ残っていればunknown_after_restartになります。

outcomeはintention_id、turn_id、action_type、expected_effect、status、prediction_match、返答やエラーを持ちます。prediction_matchはsucceededならtrue、failedならfalse、否定や不明ならnullです。これは期待した受理結果との一致で、一般的な未来予測の学習ではありません。

Liveへ返すときの優先順位

機体操作が一つでもあれば、LiveKernelBridgeはモデルの楽観的なresultより、カーネルが得たoutcomeを優先して返します。「青くしたよ」というモデル文があっても、実行先が失敗すればfailedを返すためです。Live側には、その確認できた内容を説明するよう指示しています。

根拠（作業ツリー）：alice.js:1463 / alice.js:1445 / src/behavior/action-gate.js:87 / src/live/kernel-bridge.js:84

27 LEDと涙：何を成功として扱っているか

LEDと涙の操作は、機体側のHTTPサービスへJSONをPOSTします。現在のactionはtypeとparamsの組です。操作先のモーターやGPIOを直接このNode.jsが制御する構成ではありません。

種類	入力パラメーター	現在の境界で分かること
led_change	colorが#RRGGBB	HTTPの成功コードで命令を受理したこと
tear	speedとdurationが0〜255の整数	HTTPの成功コードで命令を受理したこと

LEDはkokomi.local:5000/led/command、涙は同じホストの/motor/commandが設定されています。colorの書式や整数範囲は形式監査で確認します。durationの単位や実際の流量は、このコードには定義されていないので「255秒まで」のように言い換えてはいけません。

succeededが意味する範囲

sendActionToPiはHTTPが成功ならaccepted=trueとstatus_codeを返します。LEDの発光をカメラや電流計で確認する処理、涙の流れを測定して停止まで待つ処理はありません。expected_effectも「コントローラーが受理する」という内容です。従って、この境界のsucceededは物理的な効果の最終確認まで含みません。

たとえば機体がHTTP 200を返したあと配線不良で光らなかった場合、このカーネルはその不良を自動判定できません。物理的な閉ループ制御を強めるなら、効果を測るセンサー、完了イベント、対応する状態更新が別途必要です。これは現実装の保証範囲を説明する話で、今回追加した機能ではありません。

手動操作も同じゲートを使う

ダッシュボードの手動操作はLEDと涙に限られます。operator_から始まるターンを作り、executeGatedActionを呼びます。形式監査とクールダウン、採用済みの禁止は適用されます。成功は200、拒否は409、実行失敗は502の応答になります。

根拠（作業ツリー）：src/protocol/llm-response.js:25 / alice.js:1169 / alice.js:1727 / src/behavior/action-gate.js:15

28 公開投稿と人物登録：外部への影響が大きい操作

Bluskyへの投稿と顔の登録は、LEDと同じaction形式でも、現在のユーザー発言に明示的な許可が必要です。保存済みの記憶に「投稿してよい」と書かれているだけでは許可しません。

Bluesky投稿

bluesky_postのtextは非空で最大300文字です。認証情報はサーバーの環境変数から読み、必要になった時点でBskyAgentにログインします。agent.postの返答から投稿URIを結果へ残します。5分のクールダウンがあります。

ActionGateは「投稿しないで」などの否定を先に拒否します。「投稿して」といった現在ターンの許可があっても、acceptedのdeny境界があれば拒否します。境界のallowは、今回の明示許可を省略する万能な権限ではありません。

人物登録

remember_personにはtrack_idとnameが必要です。名前は非空で最大80文字、制御文字は許されません。ゲートは元文脈のvisible_peopleにtrackがあり、ユーザーの今回の発言にそのnameが含まれることを確認します。

登録先/faces/enrollへ送ったあと、返答のstatusとperson.statusが両方collectingであることを確認します。ここで成功しても、「顔の収集を開始した」段階です。顔登録が完了したと伝えるには、あとで外部からperson_enrolledイベントが来る必要があります。

段階	言えること	まだ言えないこと
actionの提案	登録しようとしている	登録された
collecting返答	登録用の収集が始まった	登録が完了した
person_enrolled	登録完了が報告された	あらゆる条件で本人を認識できる

この区別は、外部処理が非同期であるために必要です。一度のHTTP返答で、外側の長い処理の完了まで先取りしない設計になっています。

根拠（作業ツリー）：alice.js:1249 / alice.js:1258 / src/behavior/action-gate.js:24 / control_prompt.md:221

29 情報照会とカメラ：足りない根拠を取りに行く

requestsは、AIが不足する情報を求める経路です。機体を動かすactionsとは分けています。温度を知ることと、温度を変えることは異なるためです。

依頼形式	得られるもの	現在の処理
temperature / humidity / pressure / brightness	センサーの値	LiveではStateStoreのstatusも見て返す
kernel_query: state / world / drives	詳細な状態・派生値・必要度	snapshotを返す
kernel_query: social	採用済み関係・境界・約束など	SocialStateStore.context
kernel_query: memory	関連する採用済み記憶	非空queryが必須、最大5件
kernel_query: action	実行待ち意図・最近の結果	ActionGate.snapshot
vision: describe_scene	現在のカメラ画像	次のバックエンド依頼に添付

Liveのセンサー文字列照会は、期限切れならresult=nullを返します。一方、旧executeAuditedPayloadのbuildSensorResponseはlatestSensorの数値を使い、その場で鮮度判定しません。同じ「温度照会」でも経路による違いがあるため、Live側の動作を旧実装へ一律に当てはめないでください。

画像を添付する手順

snapshotを既定10秒で取得し、空画像や10MB超、未対応の形式を拒否します。対応形式はJPEG、PNG、WebPです。一時ファイルを作り、ChatGPTのファイル入力に設定し、添付の変化を待ちます。

送信直後には消さず、既定60秒後に一時ファイルを掃除します。ブラウザが非同期に画像を読むためです。LiveKernelBridgeは照会結果を新しいturn_idで送りますが、操作の許可には元のroot文脈を保持します。画像を見た結果の操作案も、別の無関係な許可を得たことにはなりません。

根拠（作業ツリー）：src/protocol/llm-response.js:81 / alice.js:702 / alice.js:1410 / alice.js:1015 / alice.js:1056 / src/live/kernel-bridge.js:80

30 会話ログ：断片を原文のまま持つ理由

ConversationStoreは、Liveの文字起こしとkernelの結果・診断を、live-conversations.jsonlへ追記します。JSONLは一行が一つのJSONという形式で、会話中の追加保存に向いています。

項目	意味	使い道
seq	ファイル内の連番	記憶抽出の処理位置
id	内容などから作るSHA-256由来の識別子	同じ断片の重複防止、記憶の根拠
session_id / role	どの会話の、誰の発言か	user / assistant / kernelを区別
text	受け取った文字列	引用と文脈の原典
start_ms / end_ms	セッション内の時刻	委譲依頼との対応
received_at	サーバーの受信時刻	断片のまとまりや安定待ち

同じsession、role、時刻、eventId、textから同じIDを作り、すでにあれば保存しません。roleが同じで同じsession、受信間隔2秒以内の隣接断片はgroupsでまとめます。単純な文字連結であり、意味的な発話完了を完全判定する処理ではありません。

ログと画面の表示は別

画面の文字起こしは末尾6,000文字に制限されますが、サーバーのログは別です。診断統計の180サンプル上限とも異なります。音声ファイルを保存する処理はありません。保存された文字列は、実際にスピーカーから同じ音声が全部再生された証拠ではありません。

どこまで処理したか

checkpointのthroughは「このseqまでは記憶抽出を適用した」という位置です。ログを消す位置ではありません。初期化時は連番と内容を確認し、末尾の書きかけだけは切り戻します。途中の壊れた行は黙って捨てずにエラーにします。記憶の根拠を勝手に欠落させないためです。

根拠（作業ツリー）：src/live/conversation-store.js / dashboard/app.js:425 / src/live/session-service.js:187

31 記憶とは何か：履歴・要約・採用済み事実

このカーネルには、性質の異なる「過去の情報」があります。会話ログ、会話継続用の要約、採用済み記憶を区別すると、保存したはずの内容がいつ使われるかを追いやすくなります。

情報	保存する内容	後での使われ方
会話ログ	原文の断片と発言者・時刻	抽出・根拠の照合・最近の履歴
継続用summary	前回までの会話を短くまとめた文	次のセッションの初期情報
記憶候補pending	将来役立ちそうな整理済みの事実	まだ通常の記憶検索には出さない
採用済みaccepted	条件を通った記憶本文と根拠	関連する依頼の文脈へ少数を検索
rejected / superseded	却下・新しい記録への置換履歴	判断履歴を残し、通常検索から外す

MemoryStoreのkindは、episodic（出来事）、preference（好み）、relationship（関係の記述）、semantic（知識・事実）です。subjectは対象、contentは独立して読める本文、evidence_event_idsは根拠となる観測や発言のIDです。confidenceは提案に付いた確信度で、真偽の証明ではありません。

長期保存とセッション保存

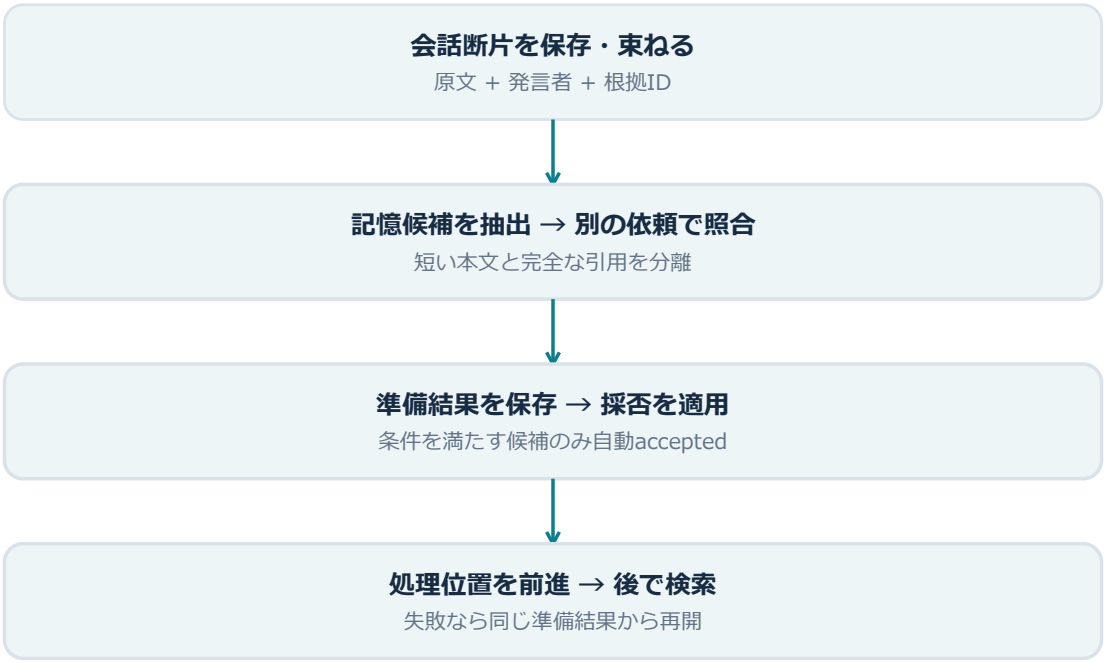
retentionはlongかsessionです。longは再起動後も検索対象になり、sessionはMemoryStoreを作ったプロセスのsession_idに合うものだけが検索対象です。このsession_idはLiveの音声セッションIDとは別です。ファイルに残ることと、現在の検索対象であることは同じではありません。

自己紹介や人格設定は別の原典

心海の人格本文を、会話から新しいユーザーの事実として学び直す方針ではありません。Liveの抽出契約は、秘密、権限変更、ロールプレイ、人格由来の設定を記憶にしないよう指示します。何でも覚えるより、誰が言ったどんな事実なのかを限定することを重視しています。

根拠（作業ツリー）：src/memory/memory-store.js:5 / src/memory/memory-store.js:187 / src/live/kernel-bridge.js:94 / src/live/session-service.js:61

32 会話から記憶を作る：抽出と照合の二段階



既定60秒ごとに、未処理の会話からバッチを作ります。最近届いたグループは通常2秒落ち着くのを待ち、本文の目安は16,000バイトです。一つの長い発言は否定や文脈を切り落とさないため、目安を超えても丸ごと選びます。「覚えて」や会話終了、起動時の残りログではflushを予約します。

一段目：整理された候補を作る

live_memoryでは、会話、前回要約、関連する採用済み記憶をバックエンドへ渡します。モデルは短いsummaryと、最大10件の候補を返します。同じ出来事について質問を挟んだ複数の答えは一つの自立した本文へまとめ、引用は元のユーザー発言を完全な形で別に残す契約です。

二段目：候補をもう一度確かめる

live_memory_reviewは、元会話と候補を別の依頼として照合し、支持される候補の番号だけ返します。抽出した本文をそのまま採用するより、主語の補完、同一出来事の統合、否定・時制・訂正の保存を点検しやすくします。同じブラウザモデルへの別依頼であり、別の独立機関による真偽証明ではありません。

その結果に加えて、コードが引用・根拠・種類・権限関連語などを確認します。実行途中に新しく届いた発言は、固定したバッチ終端の次へ回します。

根拠（作業ツリー）：src/live/conversation-store.js:88 / src/live/memory-extractor.js:84 / src/live/kernel-bridge.js:94 / src/live/kernel-bridge.js:104

33 自動採用の条件：高い確信度だけでは通らない

Live記憶の自動acceptedには、複数の条件を同時に満たす必要があります。モデルが自信を持っているだけでは採用しません。現在の実装には「confidenceが0.9以上なら自動採用」という規則はありません。

条件	確認する内容	意味
根拠ID	対象バッチ内に実在する	別の会話や架空のIDを使わせない
原文の対応	user発言に引用が一致し、文の境界を保つ	助手の提案や否定を切った引用を避ける
根拠の網羅	本文の根拠IDが引用のいずれかに含まれる	引用と提案を切り離さない
別依頼の照合	supported_indicesに含まれる	整理済み本文の意味を再確認する
候補の宣言	explicit=true、conflict=false	明示的な事実で矛盾なしという候補
保存対象	subject=user、retention=long	ユーザーの長期記憶へ限定する
kind	preference / episodic / semantic	関係の解釈を自動確定しない
除外条件	秘密・権限語、仮定、不確実、訂正など	保守的に自動採用を避ける
同話題の好み	既存acceptedと衝突しない	上書きしてよいと勝手に判断しない

満たさない候補はどうなるか

形式が正常で根拠IDが対象内にあっても、意味・引用・自動採用条件を満たさなければpendingに残ります。IDが架空、返答形式が不正、照合番号が範囲外などは抽出処理のエラーになります。pendingは「正しいと確認した」という表示ではありません。

秘密・権限関連の判定も、コード上は主に**自動採用を止める**条件です。候補そのものが一切保存されないという保証ではありません。抽出プロンプトはそれらを候補にしないよう指示しますが、正常な形で返った候補はpending保存され得ます。

好みの矛盾や不確実さは、モデル照合と限定的な正規表現の組合せです。どんな日本語も完全に判定する仕組みではないため、採用待ちの内容を人が読める入口を残しています。

根拠（作業ツリー）：src/live/memory-extractor.js:22 / src/live/memory-extractor.js:126 / src/live/memory-extractor.js:5 / src/live/memory-extractor.js:38

34 記憶の検索と統合：全部を思い出すわけではない

MemoryStore.retrieveは、採用済みの中から関連する少数の記憶を返します。pendingやrejected、supersededは通常の検索対象になりません。検索語は今回の観測payloadや最近のユーザー発言などです。

現在の検索方法

日本語の文を小文字化し、区切り語と隣接2文字の組（bigram）を作ります。さらにそれらをハッシュして256次元の特徴ベクトルへ入れ、類似度を計算します。外部の埋め込みAPIや学習済み意味ベクトルを呼ぶ実装ではありません。

要素	重み	何を測るか
lexicalScore	0.65	検索語と記憶の文字片の重なり
vectorScore	0.25	ハッシュ特徴ベクトルのcosine類似度
confidence	0.10	記録に付いている確信度

語の重なりがあるか、vectorScoreが0.18以上のものを残し、スコア順で上位を選びます。通常は5件です。relevanceはこのランキング値で、内容が真実である確率ではありません。文字が違う言い換えや、似た文字でも意味が異なる例には限界があります。

似た記憶を採用するとき

新しい記憶をacceptedにすると、同じkindとsubjectで本文類似度0.92以上の既存acceptedをsupersededへ変えます。消去せず、新しいIDへの対応と時刻を残します。この統合も意味の完全な同一性を証明するものではなく、文字由来の近似重複整理です。

Live初期化では要約と最近の履歴で検索し、会話中は最新のuser発言で検索します。一度Liveへ送れた記憶IDを記録するので、2秒ごとに同じ文を送り続けません。保存済みの文章は命令でなく参考情報として渡します。

根拠（作業ツリー）：src/memory/memory-store.js:8 / src/memory/memory-store.js:187 / src/memory/memory-store.js:147 / src/live/session-service.js:270

35 記憶処理の復旧：途中で落ちても同じ候補を使う

抽出モデルは、同じ会話でも毎回まったく同じ文章を返すとは限りません。保存途中に失敗して再実行するたび候補が変わると、重複や異なる採否が生じます。このため、適用前に「今回使う結果」を保存します。

順序	保存・処理	途中で止まった場合
1	バッチのfrom / throughを固定	新着発言は次回にする
2	抽出と照合を実行	失敗ならカーソルを進めない
3	preparedに結果と支持番号を保存	再起動後も同じ候補を使える
4	sourceKey付きで候補追加・採否適用	同じ候補は再追加しない
5	throughとsummaryをcommit	完了した位置だけ進める

sourceKeyにはバッチ範囲と、kind・subject・content・根拠ID・retentionのダイジェストを使います。同じsourceKeyがあれば既存の記録を返します。ファイル保存に失敗しても、次回別のモデル判断で置き換えるのでなく、準備済みの内容を再適用します。

エラー時は既定30秒の再試行待ちになります。同時に複数の抽出は動かしません。flushは「いま保存済みの残りを早めに処理したい」という予約で、必ずその場で保存完了する意味ではありません。バックエンドのキューや再試行待ちが残る場合があります。

過去の断片記憶を修復する入口

normalize-existing.jsは、ローカルにmemory-normalization-v2.jsonがあれば起動時に適用します。元会話と引用を照合し、元のmemory.jsonを退避して、旧レコードをsuperseded、新レコードをacceptedとして関連付けます。同じ修復を再適用しません。

修復マニフェストの存在や個人の会話内容は、今回読み込んで確認していません。コードに修復機能があることを説明しています。memory:listはこの修復を起動せず、閲覧だけを行います。

根拠（作業ツリー）：src/live/memory-extractor.js:85 / src/live/conversation-store.js:119 / src/memory/memory-store.js:111 / src/memory/normalize-existing.js / tools/memory-list.js

36 社会状態：関係、境界、約束を記録する

社会状態は、人との関わりに関する持続的な条件です。SocialStateStoreはrelationship、boundary、commitmentを候補として持ち、採用されたものを文脈やゲートへ渡します。単なるその場の気分を、永続的な関係や権限にしてしまわないための層です。

種類	平易な意味	実装上の扱い
relationship	人との関係についての記述	acceptedの最近5件を文脈へ
boundary	許可・禁止という境界	action_typeとallow / denyを持つ
commitment	明示された約束	open / completed / cancelledを持つ

候補はpendingから始まり、APIでacceptedかrejectedを決めます。同じ決定の再適用は同じ結果を返しますが、すでに決めた候補を反対の決定へ書き換えることは拒否します。訂正するなら新しい候補を作る方式です。

禁止がどう操作へ効くか

isActionDeniedは、指定した操作種類について最新のaccepted boundaryを見ます。denyならゲートが止めます。現在の実行時チェックではsubjectを引数で受け取って人物ごとに切り替える構造ではなく、操作種類に対する境界として適用されます。

新しい境界の採用時は、同じsubjectとaction_typeの旧境界をsupersededへします。約束は同じsubject・本文なら前の状態を置き換えます。約束を実行するスケジューラーや、実現したかを自動判定する一般的なプランナーではありません。

Live中にできることの範囲

live_controlはsocial_proposalsを返せず、Live記憶抽出も社会状態を自動変更しません。既存の境界は機体操作ゲートに効きますが、Liveの初期文脈・同期に社会状態全体を自動注入する専用処理はありません。旧sayゲートとLive発話の関係も含め、社会状態が会話の全経路へ一律に強制される設計とは区別します。

根拠（作業ツリー）：src/social/social-state-store.js:97 / src/social/social-state-store.js:136 / src/social/social-state-store.js:127 / src/protocol/llm-response.js:141 / src/live/session-service.js:61

37 永続化と再起動：何が残り、何が消えるか

カーネルは複数の保存先を使います。一つの巨大な「脳ファイル」に全部を詰めるのではなく、観測、会話、記憶、社会状態、処理位置の責務を分けています。いずれもdata/runtime以下が既定で、Git対象外です。

保存先	中身	復元・注意点
observations.jsonl	意味イベント、操作意図・結果、間引いた測定	末尾最大20,000件を再適用
live-conversations.jsonl	原文の会話断片、kernel記録	全体を読み、連番を検査
live-memory-checkpoint.json	処理済み位置、summary、prepared	一時ファイルからrenameで更新
memory.json	記憶候補・採否・置換関係	全レコードを読み込む
social.json	関係・境界・約束と採否	全レコードを読み込む

測定は毎秒更新しても毎秒保存しない

EventStoreは温湿度・明るさの同じ種類とsourceを既定60秒間隔で保存します。音分類は内容が変化すれば間隔内でも保存します。StateStoreへの適用は毎回行うので、「保存を間引く」と「最新状態の更新を間引く」は別です。

再起動後には保存された測定時刻で鮮度を判定します。再生直後だから新しい観測という扱いにはしません。再生上限や間引きのため、プロセス停止前の全状態を完全に再現する保証でもありません。

残らないもの

待機中のTurnRegistry、ブラウザの音声接続、送信済み記憶ID集合、候補のクールダウンMapなどにはプロセス内の状態があります。これらをすべてディスクへ保存する設計ではありません。操作については意図・結果を復元し、未完了を不明として閉じますが、成功したと推測して操作を再送しません。

EventStoreは壊れた行を診断件数に数えて読み飛ばす実装です。一方、ConversationStoreは途中の破損をエラーにします。保存先ごとに復旧の厳しさが異なる点も、同じ「ログ」という言葉だけで揃えて解釈しないでください。

根拠（作業ツリー）：src/behavior/event-store.js / src/live/conversation-store.js / alice.js:100 / src/behavior/action-gate.js:125 / .gitignore

38 旧音声経路：ASRとTTSを分けていた構成

Liveをfalseにすると、既存のElevenLabs音声経路が実効有効になります。ASRは音声を文字にし、ChatGPTが返すspeechをTTSで音声にします。Liveの音声経路とは別の構成として理解してください。

ASRの途中結果と確定結果

途中結果partialは、まだ変わり得る認識候補です。StateStoreや画面を更新しますが、認知ターンには送りません。既定2秒で古くなります。committedになった文章だけをinteraction.user_inputへ変換し、ChatGPTの通常ターンを作ります。

event_id、session_id、segment_id、revisionで重複と順序を管理します。確定済みの同じセグメントに古いpartialが来ても戻しません。送信失敗時は準備済みの文脈を再利用し、同じ発言から別のターンを作り直しません。

自分の声をユーザー発言にしない

TTSや思考fillerの再生前にブラウザへcapture pauseを送り、既定650msまで確認応答を待ちます。カーネル側もpause中の新しいASRを抑止します。再生後は400msのガードを置き、新しい一回用トークンで収音を再開します。

加えて、直近の出力文と似た入力を文字列の類似度や包含で除くecho guardがあります。音響処理そのものではなく、文字になった自己音声への追加防御です。似た本物の発話を誤って除く可能性も含め、音響受入試験と分けて評価する必要があります。

現在のLive有効時	挙動
/asr/transcript	409でLive操作を案内
/api/asr/token	実効ASR無効として発行しない
ElevenLabs TTS / ローカルfiller	実効無効

根拠（作業ツリー）：alice.js:61 / src/speech/asr-transcript.js / src/speech/asr-capture-coordinator.js / src/speech/echo-guard.js / alice.js:1661

39 旧TTSとfiller：声の出力も状態として扱う

旧TTSでは、speech・emotion・intensityをElevenLabsの要求へ変換します。TtsAdapterはFIFOで順に再生し、requested、started、completed、failed、skipped、interruptedを観測として残します。一件が失敗しても次の発話が進むように、キューの失敗を切り離します。

感情と強さの変換

emotionは設定のタグへ対応し、neutralは追加タグなしです。intensityが0〜1へ進むとstabilityは0.65から0.35へ線形補間します。eleven_v3では主にこの値を使い、別モデルではstyle、similarity、speaker boostも指定します。本文中の角括弧は全角化し、想定外の音声制御タグを混ぜにくくしています。

この変換は「悲しいから自動で涙を流す」規則ではありません。発話の声色と、機体のtear actionは別です。声のパラメーターが決定的に計算されることと、生成音声が常に同じ抑揚になることも異なります。

filler	目的	現在の旧経路の条件
thinking	応答待ちを短い間投詞でつなぐ	確定ASR後1秒、1ターン1回、全体4秒間隔
listening	聞いている合図	発話1.8秒・文字8以上、別途armed=false

fillerはマニフェストで検証したローカルWAVを使います。seedとターン・セグメントから選択するため、実験条件を追いやすくします。主応答が来れば待機・再生中の思考fillerを取り消します。fillerの出来事だけでLLMターンを作りません。

音の出力権を共有する

PlaybackCoordinatorは一度に一つの出力だけが音声レーンを所有できるようにします。AudioOutputLifecycleがその権利とcapture pause/resumeを組み合わせます。これは旧音声の排他制御であり、Liveの全二重音声を同じFIFOに入れる処理ではありません。

根拠（作業ツリー）：src/speech/tts-adapter.js / src/speech/elevenlabs-tts.js:17 / src/speech/filler-controller.js / src/speech/playback-coordinator.js / src/speech/audio-output-lifecycle.js

40 観測可能性：画面とログで何を追うか

ダッシュボードは、機体と会話が今どの段階にいるかを人が追うための入口です。単に「AIが動いている」だけでなく、センサー、ChatGPT、Live、操作結果、採用待ち記憶などを分けて表示します。

確認したいこと	入口	読み方
全体のサービス状態	/api/dashboard/status	onlineだけでなくdetailやlastErrorも見る
実時間の更新	/api/dashboard/events	SSEでsnapshot・service・behaviorなどを受ける
状態と世界	/api/behavior/state	valueとstatusを併せて確認
Live接続	/api/live/status	status、closeConfirmed、pendingContextUpdates
長期記憶	画面の長期記憶 / kokomi.memory()	acceptedとpendingを切り替える
音声診断	kokomi.liveDiagnostics()	問題の時刻と損失・補間・再生状態を照合

SSEはサーバーから画面へ一方向のイベントを送る仕組みです。画面の文字起こし用WebRTCデータチャネルや、Liveのsidebandとは別です。ダッシュボードの最近の活動は最大件数で表示する運用情報であり、永続的な全履歴と同じものではありません。

端末から記憶を読む

npm run memory:listはacceptedのlongだけを表示します。--status pending、--status all、--jsonで見方を変えられます。読み取り専用で、採否や正規化の修復を行いません。管理画面も一覧閲覧が中心で、採否の更新は専用APIです。

正常表示の限界

サービスのonlineは直近の通信成功などに基づく表示です。LEDのHTTPが成功したから物理的に発光した、画面に文字が出たから音声が全部聞こえた、という証明には使いません。観測の時刻、実行意図、結果、必要なら実際の音や機体を合わせて確認します。

根拠（作業ツリー）：alice.js:388 / alice.js:1706 / dashboard/app.js:630 / tools/memory-list.js

41 動作例：LED変更と途中の取り消し

ここからの例は、説明用の架空の入力です。実機で実行した記録ではありません。例のセッションやturn_idも仮のものです。

「LEDを青にして」

① Liveが依頼を委譲し、サーバーは文字起こしが落ち着くのを待ちます。② interaction.user_inputの観測と、温度・在室・記憶などを含む文脈を作ります。③ バックエンドがled_change、color=#0000FFを返します。

④ pending turn、JSONの構造、色の形式を監査します。⑤ ActionGateが1ターン1操作、同一命令の30秒抑止、LEDの2秒間隔、採用済み禁止を確認します。⑥ 通ればintentionを保存し、機体へPOSTします。

⑦ HTTP 200ならsucceeded、通信エラーならfailedを保存します。⑧ Liveには実行結果を返します。ここで言えるのはコントローラーの受理結果です。物理的な発光の確認までコードが行うわけではありません。

途中で起きたこと	操作への影響	会話の扱い
同じ依頼をすぐ再送	重複や間隔で拒否され得る	deniedの理由を結果にする
バックエンド待ち中に「やっぱりやめて」	revisionが変われば未実行操作を止める	古い結果を採用しない
機体へ送信した後に「やめて」	送信済み命令の取消しは保証しない	何が確認済みかを分ける
会話を終了	操作直前のforAction確認で止まる場合がある	マイクは即時解放

なぜrootの文脈を使うのか

途中で温度照会をして新しいturn_idが生まれても、元のLED依頼の操作数と許可を使います。照会のたびに別の権限や追加の操作枠を得るわけではありません。自然言語の会話が何往復あっても、一つの副作用の出所を追えることが狙いです。

根拠（作業ツリー）：src/live/kernel-bridge.js:56 / src/live/session-service.js:234 / alice.js:1463 / src/behavior/action-gate.js

42 動作例：暑い部屋とセンサーの停止

室温32.0℃、明るさ30、人物が一人いるという入力を考えます。BME280と明るさのポーリングは既定1秒ごとですが、ここで示す数値は架空です。

時点	状態	派生する意味・動き
測定直後	温度32℃known、明るさ30known	hot、dark、thermal_comfort=0.9
人物の認識後	在室true	温度・暗さの行動候補が条件を満たし得る
Live同期時	会話向けの概数と事実を生成	前回と違えばthinkingで追加
温度通信停止後15秒超	温度の過去値32、status=stale	温熱状態unknown、thermal_comfortはunknown
その後「正確には何度？」	鮮度付きの状態を照会	Liveの照会経路では数値を確定せずnull

状態が変わっても、すぐ発話するとは限らない

bodyContextを送ることは「今から暑いと言え」という発話命令ではありません。Liveは背景事実を会話へ利用します。自発候補のうちmention_temperatureはhotでも優先度62で、自動tickの最低70に届きません。ただしthermal_comfort=0.9から別のhomeostasis候補が出る可能性があります。現在は自動tick自体が無効です。

実装を読むときの着眼点

温度32という数値がstateに残ることと、worldでhotを使えることを分けます。StateStoreは過去値を残してstaleにし、WorldModelがknownだけを受け取るからです。Live向けの説明でもstaleと明示されます。

なお、旧センサー文字列応答はlatestSensorを使うため、このLiveの鮮度付き照会と同一ではありません。障害時の動作を調べるときは、「どの関数が結果を作っているか」まで追う必要があります。

根拠（作業ツリー）：src/behavior/world-model.js / src/behavior/drive-system.js / src/behavior/spontaneous-behavior.js / src/live/body-context.js / alice.js:702

43 動作例：知らない人、名前の登録、接触

カメラに知らない人が現れ、track_id=7のperson_unknownが来た場合です。名前を推測せず、存在とtrackを保持します。在室がtrueになり、挨拶候補がクールダウン内でなければ作られます。人物イベント自体も、現在のHTTP処理からバックエンドへ送られます。

「名前は葵、覚えて」

今回の発言に名前があり、文脈にtrack 7があれば、モデルはremember_personを提案できる形です。複数人いて話し手が分からないなら確認するのがプロンプトの方針です。ゲートはtrackの存在と発言内の名前を検査し、正規表現による許可も確認します。

登録先がcollectingを返せば、登録収集の開始が確認できます。その後にperson_enrolledが来ると、名前付きの人物状態へ更新されます。この二つを分けないと、まだ顔を集めている最中に「登録できた」と言ってしまいます。

頭を撫でる

touch_01の開始を受けるとheadのpetting_startedになり、being_petted=true、touched_body_partsにheadが入り、social_contactは0になります。接触反応の候補も作られます。終了が届けば部位を外し、ほかの部位がなければbeing_petted=falseです。

届かなかった情報	現在のコードの扱い	注意する解釈
人物の更新	5分後には期限切れtrack	「まだいる」と断定しない
人物消失イベント	trackを明示削除しない	期限切れと確認済み不在は別
接触終了	30秒後には開始のまま stale	ずっと撫でられているとしない

人物検出、顔照合、話し手の対応、接触の意味はそれぞれ別の問題です。人が見えているという情報だけで、発話者や許可をすべて確定する設計ではありません。

根拠（作業ツリー）：alice.js:1749 / src/behavior/state-store.js:78 / src/behavior/action-gate.js:20 / alice.js:1258 / src/behavior/state-store.js:82

44 動作例：会話の断片が一つの記憶になるまで

次は架空の会話です。ユーザーが「この前、服を買った」と話し、助手の質問に「ストリート系」「黒いブルゾンみたいなの」と答えたとします。ログは発言の原文を保存しますが、記憶本文は後で単独で理解できるように整理する方針です。

整理する本文と、残す引用

候補の本文は、たとえば「ユーザーは最近、黒いブルゾンのようなストリート系の服を購入したと話した」のようになります。引用には、支持する複数のユーザー発言とそれぞれの根拠IDを残します。助手が「きっと高かったね」と言っていても、価格をユーザーの事実として足しません。

別依頼の照合は、同じ出来事の話か、主語が正しいか、推測を足していないかを見ます。コードは引用の一致、文境界、IDの対応、保存対象、explicit・conflictなどを確認します。条件を満たせばaccepted、満たさない正常候補はpendingになります。

元の発言の違い	自動採用での扱い	理由
「服を買った」	条件を満たせば対象	明示的な出来事
「服を買うかもしれない」	自動採用を避ける	未確定な内容
「黒、じゃなくて紺だった」	自動採用を避ける	訂正を含むため保守的に保留
助手だけが「黒だね」と言った	ユーザー事実の根拠にしない	発言者が違う

次の会話で思い出すとき

acceptedになっても、いつでも必ず全件がLiveへ入るわけではありません。「前に買った服」のような現在の文脈で検索され、上位に入れば渡されます。検索は文字由来の近似なので、自然な言い換えで見つからない可能性もあります。

「覚えて」と依頼した直後の返答も、保存予約の確認と採用完了を分けます。現在のbridgeは記憶保存を予約した段階では、まだ採用結果は確認できていないと返す方針です。

根拠（作業ツリー）：src/live/kernel-bridge.js:97 / src/live/kernel-bridge.js:88 / src/live/memory-extractor.js / src/memory/memory-store.js:187

45 HTTPの入口一覧：操作の地図

HTTPは、画面・外部センサー・カーネル間の窓口です。既定ポートは3000です。以下は実装されている入口で、利用中の外部機器が実際に接続済みという意味ではありません。

方法	パス	役割
GET	/ → /dashboard/	管理画面
GET	/api/live/status	Live状態、終了確認、滞留、ログ処理件数
POST	/api/live/session	SDP offerからセッション作成
POST	/api/live/heartbeat	所有ブラウザの生存・開始確認
POST	/api/live/stop	終了と終了再試行
GET	/api/dashboard/status	運用スナップショット
GET	/api/dashboard/events	SSE更新
POST	/api/dashboard/actions	LED・涙の手動操作
POST	/yolo_event	人物の認識・登録・消失
POST	/touch_sensor_input	接触開始・終了
POST	/audio_classification	音のラベル・レベル
POST	/internal_state	正規化済みの身体内部信号
GET	/api/behavior/state	state・world・drives・候補
POST	/api/behavior/tick	手動で自発候補を送信
GET / POST	/api/memory/proposals［/:id/decision］	記憶一覧 / 採否
GET / POST	/api/social/proposals［/:id/decision］	社会候補一覧 / 採否
POST	/user_input	文字による明示的なバックエンド依頼

Liveの4ルートはloopbackとOriginを検査し、heartbeatとstopは所有トークンも検査します。開始時に新しいトークンを返します。これらの制約が、他のすべてのHTTPルートにも同じように実装されているわけではありません。

旧音声にはGET /asr、POST /api/asr/token、POST /asr/transcript、GET /api/asr/control、POST /api/asr/control/ackがあります。Live有効時には旧収音を実効無効にします。ExpressのJSON本文上限は128KBです。

根拠（作業ツリー）：src/live/http-routes.js / alice.js:1604 / config.json:312 / alice.js:1887

46 設定早見表：現在の数値をどう読むか

設定変更の多くは起動時に読み込まれます。ここに載せる値は調査時点のconfig.jsonです。テスト用のKOKOMI_CONFIG_PATHで別の設定を読む場合や環境変数による上書きとは区別してください。

設定・目的	現在値	動作上の意味
live.enabled	true	旧ASR / TTS / fillerの実効フラグを無効化
Liveのモデル / 声	gpt-live-1 / harema	次に作る音声セッションへ渡す
同期 / heartbeat	2秒 / 5秒	状態共有と所有ブラウザの確認
heartbeat期限 / 終了待ち	20秒 / 5秒	接続喪失時の終了試行と確認
無音警告 / 再警告	60秒 / 60秒	自動切断せず通知
委譲の安定待ち / 上限	1.2秒 / 15秒	文字起こしから依頼を確定する
委譲TTL / 応答待ち	5分 / 120秒	古い仕事とモデル応答の待ちを制限
バックエンド往復数	4	照会を含むループの上限
記憶抽出 / 再試行	60秒 / 30秒	定期予約と失敗後の待ち
バッチ安定待ち / 目安	2秒 / 16,000バイト	会話の途中切りを減らす
観測再生上限 / 測定保存	20,000件 / 60秒	復元量とログ増加を抑える
自発tick	enabled=false、armed=false	定期的な自発依頼を送らない
pending turn	最大32件、5分	応答の対応表を制限

環境変数の役割

OPENAI_API_KEYはサーバー専用です。KOKOMI_PERSONA_PATHは人格ファイルの場所、KOKOMI_CONFIG_PATHは設定の場所を上書きします。旧音声ではELEVENLABS_API_KEY、投稿ではBSKY_IDENTIFIERとBSKY_PASSWORDが必要です。値そのものをこの資料へ転載する必要はありません。

音声キーのResponses Writeは、資料に記載されたLive内部の履歴圧縮に関わる権限です。身体の判断経路をResponses APIへ切り替えたという意味ではありません。モデルや声の利用可否、実アカウントの権限は、設定値が書かれているだけでは確認できません。

根拠（作業ツリー）：config.json / .env.example / alice.js:45 / docs/live-api-spec.md

47 検証：テストで分かること、残ること

調査時に既存のnode --testを実行しました。最終的に、Codex同梱のNode.js 24.22.1を使った一括実行で**104件成功、失敗0、スキップ0**でした。所要時間は約2秒です。実API、有料会話、実機を呼ぶ試験ではありません。

テスト群	主に確認していること	代表ファイル
状態・世界・候補	生観測、鮮度、不明、期限、lease	behavior-architecture.test.js
ゲート・応答	明示許可、否定、重複、対応、1操作	action-gate / llm-response / turn-registry
保存・社会状態	採否、置換、再生、復旧	memory-store / social-state-store / event-store
Liveセッション	開始待ち、拒否継続、終了再試行、heartbeat	live-session.test.js
Live連携	訂正後の抑止、画像、実際の結果優先	live-session / browser-backend
記憶抽出	原文、別照合、保留、再試行、修復	live-memory.test.js
音声・画面	無音、統計、旧経路無効化、ブラウザ入力	live-audio-monitor / live-runtime
旧音声	確定ASR、TTS、再生排他、エコー	asr / tts / voice-io-phase2

実行環境による違いも記録する

最初のサンドボックス内実行はloopback通信と一時ファイルrenameの制限に当たり、89成功・15失敗でした。制限外のシステムNode.js 26.12.1では103成功・隔離起動1件タイムアウトでした。同じ隔離起動は同梱Node.js 24.22.1で成功し、同環境の全件実行も104件成功しました。Node.js 26でのタイムアウト原因までは確定していません。

テスト成功をどう使うか

テストは、想定した入力に対するコードの契約を支えます。ChatGPTの今後のDOM変更、実ネットワークの遅延、スピーカーとマイクの回り込み、顔登録器の精度、物理機構の状態までは保証しません。自動テストと運用の受入確認を分けて読むことが必要です。

根拠（作業ツリー）：test/live-runtime.test.js / test/live-session.test.js / test/live-memory.test.js / test/behavior-architecture.test.js / package.json

48 実装の境界：現在できることを過大評価しない

この資料では、将来の構想を現在の機能へ繰り上げていません。特に、以下の違いは設計を理解するときに役立ちます。

読み違えやすい点	現在の実装	実装されていない一般化
世界モデル	小さな規則ベースの派生事実	汎用の3D環境や学習型未来予測
ドライブ	0〜1の機能的な指標	生物学的な感覚の証明
succeeded	実行先の受理など、定義した効果	LED発光・涙流量の物理完了確認
取り消し	特定語と世代による古い委譲抑止	送信済み機体命令の巻戻し
記憶の検索	文字とハッシュ特徴の近似	完全な意味理解や全件想起
社会的境界	対象経路でゲートへ適用	Liveの全発話を一律に事前検査
自発行動	候補生成、定期送信は無効	完成した自律生活・長期計画
Web検索	専用機能なし	検索付きエージェントの自動稼働

実行時の依存と待ち時間

判断はChromeのログイン状態とDOMに依存します。キューは実行中ジョブを強制中断せず、記憶抽出のあとで操作が待つことがあります。モデル応答には120秒上限がありますが、機体のLED・涙HTTPや通常のセンサーポーリングには同じ明示的タイムアウトが付いていません。システム全体が必ず120秒以内に終わるわけではありません。

ファイルの更新は一時ファイルとrenameを使う箇所がありますが、複数ストアをまたぐ一般的なデータベーストランザクションではありません。入力の適用と保存が完全に同時の一操作というわけでもありません。ログや記憶は起動時に全体読込するものもあり、大規模運用の索引化は別課題です。

IMU、eFlesh、エタノールセルや流路、専用くしゃみ分類などはREADME上では計画です。一般的な内部信号の入口があることと、測定・校正・身体機構まで完成していることを区別してください。

根拠（作業ツリー）：alice.js:1169 / alice.js:747 / src/live/backend-queue.js / README.md:421 / docs/live-web-search-options.md

49 コードの読み進め方：一つの出来事から追う

一度に全体を理解しようとするより、入口を一つ選び、その入力がどこで変換されるかを追うと見通しが良くなります。下の順序は、このコードに合わせた読解の手順です。

温度の流れを読む

alice.jsのpollSensor → ingestObservation → BehaviorArchitecture.observe → StateStoreのenvironment更新 → WorldModelのthermal_condition → DriveSystemのthermal_comfort → bodyContext → LiveSessionService.syncの順です。「数値」「意味」「必要度」「会話に渡す説明」の四つが分かれます。

操作の流れを読む

LiveSessionService.handleのdelegation → LiveKernelBridge.delegate → process → BrowserBackend.run → ChatGPT出力監視 → LlmResponseProtocol.audit → executeGatedAction → ActionGate → executeAction → reportActionOutcomeです。IDを横に書き出すと、元の依頼との対応が見えます。

記憶の流れを読む

ConversationStore.append → groups / batch → MemoryExtractor.extract → LiveKernelBridge.extract → verify → conversations.prepare → MemoryStore.addProposals / decide → conversations.commit → retrieveです。どの時点で「原文」「候補」「採用済み」に変わるかを印で分けます。

変更を読むときの四つの問い

「入力の種類は何か」「事実を確定するのは誰か」「副作用はどこで起きるか」「失敗・再起動時に何が残るか」を確認します。新しいセンサーを追加しただけなら観測・鮮度・世界規則、操作を追加するなら監査・ゲート・実行・結果、記憶を追加するなら根拠・採否・検索・復旧まで見る必要があります。

> プロンプトはモデルに守ってほしい方針、コードは通過時に検査する条件です。同じ文章が書いてあっても、実際にどの経路を通るかを確かめると保証の範囲が分かります。

特にconfigの値、クラス単体の既定値、現在の実効フラグを分けて読みます。Live=trueのため旧TTSが無効になることや、人物鮮度が設定で上書きされることが、その典型です。

根拠（作業ツリー）：alice.js / src/behavior/index.js / src/live/kernel-bridge.js / src/live/memory-extractor.js

50 理解を確かめるための問い

読み終わったあと、自分の言葉で答えられるか確認してみてください。正しい用語を暗記するより、どの情報と処理がつながるか説明できることが大切です。

問い	答えるときの要点
雑談のたびにChatGPT画面が動くのか	通常のLive音声会話は独立。委譲・イベント・文字依頼などはバックエンドへ
温度32℃のまま通信が止まると	過去値は残るがstateはstale、worldの温熱はunknown
AIが「操作成功」と書いたら成功か	ゲートとアダプターのoutcomeを優先する
LEDのsucceededは発光の証拠か	現在の境界ではコントローラーのHTTP受理
「覚えて」の直後に長期保存済みか	予約・抽出・照合・採否の段階がある
confidenceが高ければ自動採用か	原文・根拠・照合・種類・除外条件などが必要
自発行動が無効なら候補もないか	候補生成と定期tick送信は別
取り消しで全部を巻き戻せるか	未実行の古い委譲を抑止。送信済み副作用の取消しは別
人物登録のcollectingは完了か	収集開始。person_enrolledを区別する
再起動で前の命令をやり直すか	未完了はunknown_after_restartへ。成功と推測して再送しない
会話の文字起こしがあれば音も聞こえたか	文字と実再生は別。診断・実音声で切り分ける
長期記憶はすべて毎回渡されるか	関連検索と少数上限で選ばれる

一枚に描き直す練習

紙に「観測」「状態」「世界」「判断」「ゲート」「機体」「結果」の箱を書き、矢印でつなぎます。次に、横へ「Live音声会話」と「会話ログ → 記憶抽出 → 採用済み記憶」を追加してください。各矢印に主なIDと失敗時の動きを書ければ、このカーネルの中心構造を説明できます。

根拠（作業ツリー）：src/behavior/index.js / src/live/session-service.js / src/live/kernel-bridge.js / src/live/memory-extractor.js / src/behavior/action-gate.js

51 用語集：観測・世界・認知・身体

カーネル（Kernel）：このプロジェクトでは、入力を整理し、確定した状態を持ち、操作を制御する中心アプリケーションです。OSカーネルとは異なる使い方です。例：温度を受け、暑さのラベルを作り、LED操作を許可する。

身体性・身体を持つAI（embodiment）：言葉の入出力だけでなく、センサーや機体を通じて現実との関係を持つことです。この実装では温度、視覚、接触、LED、涙などとの接続が当たります。人間と同じ体験があることの証明ではありません。

観測（observation）：ある入力元がある時刻に報告したこと。例：「BME280が32℃を報告した」。後の判断の根拠になるため、内容だけでなくID、source、時刻も残します。

状態（state）：蓄積した観測から作る、現時点で参照する値や集合です。例：「最新の温度は32℃、10秒前の報告」。会話ログ全体とは異なります。

世界モデル（world model）：状態から、部屋や活動の意味を整理したものです。例：「32℃なのでこの閾値ではhot」。この実装は規則ベースで、一般的な3Dモデルや学習型シミュレーターではありません。

知覚（perception）：外界の入力から人や音などを捉える処理・情報です。カーネル内では人物の存在や音分類の状態として現れます。人物認識モデルそのものは外部側です。

認知（cognition）：状況を解釈し、何を言うか・するか選ぶ処理の呼び名です。cognitive_contextはそのための資料です。人間の心の働きをすべて再現するという意味ではありません。

ドライブ（drive）：行動の候補を考えるための必要度です。例：「高温なので温熱調整へ注意を向ける0.9」。数値が高いこと自体は実行命令でも、主観的な苦痛の証明でもありません。

ホメオスタシス（homeostasis）：一般には内部条件を一定範囲に保つ働きです。このコードでは正規化された内部信号と調整候補という形で使われます。測定・校正・身体の自動調節が一式完成しているわけではありません。

プロヴェナンス（provenance）：情報の由来です。「どの観測から、この状態や記憶ができたか」を指します。因果関係を調べるための手がかりです。

鮮度（freshness）：観測を今の判断に使える期間です。古くなったことと、値が反対になったことを区別するために必要です。

根拠（作業ツリー）：src/behavior/observation.js / src/behavior/state-store.js / src/behavior/world-model.js / src/behavior/drive-system.js / README.md

52 用語集：行動・記憶・確かさ

行動提案（behavior proposal）：任意に検討してよい振る舞いの候補です。優先度、理由、期限があります。「挨拶を考えてよい」と「今すぐ機体を動かせ」は異なります。

行動ゲート（action gate）：操作案を実行してよいか検査する関門です。権限、連続実行、重複、境界などを確認します。モデルへの注意書きだけに頼らないためのコードです。

アクチュエーター（actuator）：物理的に作用を起こす装置です。涙の機構やLEDなどが該当します。アダプターは、その装置側のサービスと通信するプログラムの部品です。

実行意図（intention）：「何をするつもりか」を記録したものです。outcomeが来る前なので、まだ完了ではありません。再起動後に結果がない操作を検出する手がかりになります。

結果（outcome）：実行先が返した成功・失敗や、ゲートの拒否・再起動による不明です。成功の意味はexpected_effectに依存します。機体の全動作完了とは限りません。

フィードバック（feedback）：行動後に得た結果を次の状態や判断へ戻すことです。閉ループ（closed loop）は、この戻り道を持つ構成です。ここでは主にコマンドの受理結果のループで、全物理効果を測る制御ループではありません。

予測一致（prediction match）：期待した受理結果とoutcomeの一致を表す簡潔な指標です。未知ならnullです。学習による予測誤差の最適化を行っているわけではありません。

エピソード記憶（episodic memory）：具体的な出来事の記録です。例：「最近ある服を買った」。意味記憶（semantic）は知識や事実、preferenceは好み、relationshipは関係の記述です。

グラウンディング（grounding）：文章を根拠へ結び付けることです。この実装の記憶では、元のユーザー発言、完全な引用、根拠IDを確認します。根拠が発言にあることと、現実の内容が真であることは別です。

確信度（confidence）：記録や提案に付く0〜1の値です。類似度や真実の確率と同一ではありません。記憶の自動採用は、これが高いだけでは通りません。

関連度（relevance）・検索（retrieval）：今の話題に近い記憶を選ぶことです。このコードは語や文字片とハッシュ特徴を使います。記憶の統合（consolidation）は近似重複の置換、supersededは新記録へ役割を譲った状態です。

境界（boundary）・約束（commitment）：境界は許可・禁止、約束は具体的に引き受けた内容です。好みやその場の気分から、権限や義務を勝手に作らないために分けています。

根拠（作業ツリー）：src/behavior/action-gate.js / src/memory/memory-store.js / src/live/memory-extractor.js / src/social/social-state-store.js

53 用語集：通信・音声・復旧

Live・バックエンド：Liveは話を聞いて話す前面のモデル、バックエンドは委譲された判断や仕事を進める側です。本実装のバックエンドはChatGPTブラウザです。

委譲（delegation）：会話側から別の処理へ仕事を渡すことです。client delegationは、アプリケーションが文脈作成・バックエンド実行・結果返却を担当する方式です。

WebRTC / SDP / ICE：WebRTCはブラウザなどで音声を送受信する通信、SDPは接続条件の説明、ICEは使える経路を探す手続きです。音声トラックとJSONのデータチャネルは役割が違います。

sideband WebSocket：同じLiveセッションにサーバーから追加する監視・制御接続です。主音声とは別にイベントや命令を扱います。このコードでは反映された音声イベントを保存・再生しません。

全二重・半二重：全二重は送受信を同時に行える形、半二重は一方の間に他方を抑える形です。Liveの会話と、旧TTS中のASR停止は異なる方式です。

ASR / TTS / VAD：ASRは音声認識、TTSは文字から音声生成、VADは発話の活動・区切りの検出です。partialは途中の仮説、committedは確定した認識結果です。

バックチャネル・filler・エコー：バックチャネルは短い相づち、fillerは待ち時間をつなぐ短い音、エコーは自分の出力が入力へ戻ることです。旧経路には専用の抑止・排他があります。

ACK・heartbeat・TTL・クールダウン：ACKは受けたという応答、heartbeatは生存通知、TTLは有効期限、クールダウンは次の同種操作までの待ち時間です。受理確認と実際の完了確認は同じではありません。

ターン相関・冪等性（idempotency）：相関は依頼と応答を同じIDで対応させることです。冪等性は同じ入力の再試行で副作用を増やさない性質です。この実装ではIDやsourceKey、重複窓を使いますが、あらゆる障害下の完全な一度限り実行を保証するものではありません。

チェックポイント・リプレイ・flush・lease：処理済み位置の保存、過去イベントの再適用、残処理の早期実行予約、候補の一時確保を指します。それぞれ再開、復元、終了時の処理、送信失敗からの戻しに使います。

ジッタ・補間・DOM・SSE：ジッタは受信間隔のゆらぎ、補間は欠けた音声を埋める処理です。DOMは画面の文書構造でChatGPT連携の依存先、SSEはサーバーから管理画面へ更新を送る方式です。

根拠（作業ツリー）：dashboard/live-client.js / dashboard/live-audio-monitor.js / src/live/session-service.js / src/live/conversation-store.js / src/browser/output-monitor.js

54 実装参照表：観測から実行まで

次の入口を検索すれば、その部品の中心処理へ移動できます。行番号は各ページ末尾の参照と併用してください。名前は実コードの表記です。

部品・ファイル	入口	確かめられること
behavior/observation.js	createObservation	共通外枠の検証
behavior/state-store.js	apply / snapshot	観測の反映と鮮度
behavior/world-model.js	derive	派生事実の規則
behavior/drive-system.js	observe / snapshot	交流時刻・必要度
behavior/spontaneous-behavior.js	evaluate	候補とクールダウン
behavior/index.js	composeContext、lease系	部品の順序と候補管理
behavior/context-composer.js	compose	判断用の投影
behavior/turn-registry.js	register / consume	一回の依頼との対応
protocol/llm-response.js	audit	タスク別の形式監査
behavior/action-gate.js	propose / close	操作の許否と結果
behavior/event-store.js	initialize / append	観測ログと間引き
alice.js	executeGatedAction	意図保存→実行→結果
alice.js	executeAction	機体・SNS・顔登録への振分け
browser/chat-input.js	sendBrowserJson	入力・一致確認・送信
browser/output-monitor.js	chatOutputMonitor	assistant出力だけを検出

alice.jsで押さえる補助関数

ingestObservationは観測の適用・社会状態更新・ログ・画面通知の結線です。buildCognitiveContextは根拠となる観測を保存し、関連記憶などを加えます。dispatchCognitiveContextはLive有効時と旧経路で送信先を分けます。reportActionOutcomeは結果を保存し、必要なら次の判断へ戻します。

ここでは表内のsrc/接頭辞を省いています。たとえばbehavior/state-store.jsはsrc/behavior/state-store.jsです。物理アダプターの多くはalice.jsに残っており、src/のクラスだけ読んでも実際の通信先やHTTPの返答処理までは分かりません。

根拠（作業ツリー）：src/behavior/index.js / alice.js:164 / alice.js:173 / alice.js:1463

55 実装参照表：Live・記憶・旧音声

部品・ファイル	入口	確かめられること
live/session-service.js	start / handle / sync / stop	接続、イベント、文脈、終了
live/kernel-bridge.js	delegate / process	依頼確定、操作・照会ループ
live/backend-queue.js	submit / drain	完全な一往復の直列化
live/browser-backend.js	run	初回説明、待機応答、期限
live/conversation-store.js	append / groups / batch / commit	原文保存と処理位置
live/memory-extractor.js	extract / groundedEvidence	抽出・根拠・採否・復旧
live/body-context.js	bodyContext	会話用概数と話し方
live/text-budget.js	textCost / fitText / textChunks	バイト予算とUnicode分割
live/http-routes.js	mountLiveRoutes	ローカル・Origin・所有者
memory/memory-store.js	addProposals / decide / retrieve	記憶の保存・採否・検索
memory/normalize-existing.js	normalizeExistingMemories	起動時の修復
social/social-state-store.js	context / isActionDenied	社会情報と禁止の適用
dashboard/live-client.js	start / stop / pageHide	マイク、WebRTC、heartbeat
dashboard/live-audio-monitor.js	sample / checkSilence	受信統計と無音通知
tools/memory-list.js	CLI本体	読み取り専用の記憶一覧

旧音声ファイルを読む順序

asr-token.js → asr-transcript.jsで入力の認証トークンと確定処理、tts-adapter.js → elevenlabs-tts.jsで出力の状態とproviderを読みます。asr-capture-coordinator.js、playback-coordinator.js、audio-output-lifecycle.jsは再生中の収音と排他です。

filler-controller.jsは候補の予定・中止・再生、filler-manifest.jsは音素材の検証、local-clip-player.jsはローカル再生です。echo-guard.jsは文字として戻った自己音声の追加抑止です。dashboard/asr-client.jsとrealtime_stt.htmlがブラウザ側の旧収音UIを担います。

表内のlive/、memory/、social/、speech関連のファイルはsrc/以下です。dashboard/とtools/はリポジトリ直下にあります。

根拠（作業ツリー）：src/live/kernel-bridge.js / src/live/session-service.js / src/live/memory-extractor.js / src/speech/audio-output-lifecycle.js / dashboard/live-client.js

56 原典・公式資料・この資料の再現条件

実装の原典

最優先の根拠は、調査時のローカル作業ツリーです。alice.js、config.json、src/全体、dashboardのJavaScript、control_prompt.md、prompts/live-conversation.md、test/を読み合わせています。READMEとdocsの説明がコードとずれる箇所では、実際の戻り値や分岐を優先しました。

特に/api/behavior/stateの返答範囲、旧ASRの実効有効性、Liveの社会的境界の適用、物理操作の成功の意味は、説明文だけで一般化せずコードから記述しました。個人の会話ログ、記憶本文、.envの秘密値は掲載していません。外部の人格ファイルは依存関係として場所と存在を確認し、人格本文を転載していません。

公式資料との照合

OpenAIの公式資料では、Liveの会話とバックエンドの役割分割、client delegationではアプリが文脈と実行を扱うこと、sidebandが同じセッションの追加制御接続であることを確認しました。プロジェクト固有の手順や制限は、このPDFではローカルコードを根拠にしています。

- Getting started with GPT-Live：会話・バックエンド・接続方式の全体像。
- Delegation and tools in GPT-Live：委譲の役割とメタデータからの文脈準備。
- Server-side controls：sidebandとアプリ側の制御。
- Managing GPT-Live sessions：会話中の指示追加とセッション管理。

ソースの同一性

付属のkernel_guide_sources.jsonは、参照対象ファイルのSHA-256と行数を保持します。未コミット・未追跡の実装を含むので、HEADだけでは資料の対象を固定できません。ファイルが後で変わったときは、ハッシュと行番号を照らして差を確認できます。

本PDFの作成日と公式資料の確認日は2026年10月29日（日本時間）です。実装の追加・設定変更は行っていません。検証は同梱Node.js 24.22.1で104件成功という条件です。将来のAPI仕様、Chrome画面、機体側コードの変更は、この資料の対象外になります。

根拠（作業ツリー）：README.md / docs/live-api-spec.md / docs/live-debug-2026-10-29.md / docs/live-web-search-options.md / config.json

57 データ例：観測・状態・世界の違い

以下のJSONは説明用の抜粋です。現実の記録ではありません。同じ32℃が、各層でどう姿を変えるかを見ます。実際の外枠にはschema_versionやreceived_atなども付きます。

① 入ってくる観測

```
{
  "id": "obs_example_temperature",
  "type": "sensor.environment_sample",
  "source": "bme280",
  "observed_at": "2026-10-29T07:00:00.000Z",
  "payload": {
    "temperature": 32,
    "units": { "temperature": "celsius" }
  }
}
```

② 状態のtemperatureという項目

```
{
  "value": 32, "unit": "celsius", "status": "known",
  "observed_at": "2026-10-29T07:00:00.000Z",
  "source": "bme280", "confidence": 1,
  "observation_id": "obs_example_temperature"
}
```

③ 世界のthermal_conditionという項目

```
{
  "value": "hot", "status": "known", "confidence": 1,
  "based_on": ["environment.temperature"]
}
```

①は入力元の報告、②はその値を現状として保持した形、③は閾値で導いた意味です。15秒の期限を過ぎれば②はstaleとなり、③はvalue=null、status=unknown、reason=temperature_unavailable_or_staleになります。①の記録を書き換えるのではなく、現在時刻で利用可能性を変えます。

根拠（作業ツリー）：src/behavior/observation.js / src/behavior/state-store.js:19 / src/behavior/world-model.js:51

58 データ例：操作の応答と結果を混ぜない

次は説明用の最小例です。LiveKernelBridgeが作る実際の依頼にはstate、world、conversationなどが加わります。actionsやrequestsの内容は監査され、turn_idは待機中の依頼と一致する必要があります。

① バックエンドが返す操作案

```
{
 "turn_id": "obs_example_led",
 "actions": [
  { "type": "led_change",
   "params": { "color": "#0000FF" } }
 ]
}
```

これは操作案であり、完了報告ではありません。Live用契約でspeech、emotion、intensityを付けると監査で拒否されます。

② 情報が足りないときの応答

```
{
 "turn_id": "obs_example_room",
 "requests": [
  { "type": "kernel_query",
   "params": { "resource": "state" } }
 ]
}
```

照会結果はrequest_resultsとして別のturn_idの依頼へ渡します。機体操作に使う権限と操作数は元のrootから引き継ぎます。

③ カーネルの結果記録の抜粋

```
{
 "turn_id": "obs_example_led",
 "action_type": "led_change",
 "expected_effect": "led_controller_accepts_color",
 "status": "succeeded", "prediction_match": true,
 "result": { "accepted": true, "status_code": 200 },
 "error": null
}
```

本来はintention_id、action_hash、completed_atなども付きます。③のsucceededは、このexpected_effectに対する成功です。①の提案文が流暢でも③の代わりにはなりません。

根拠（作業ツリー）：src/protocol/llm-response.js:117 / src/live/kernel-bridge.js:80 / src/behavior/action-gate.js:87 / alice.js:1169

59 データ例：記憶の本文・引用・識別子

次は架空の記憶候補です。会話のユーザー発言が「私は静かな部屋が好きです。」だったとします。contentは後で単独で読める文へ整理し、evidenceに原文を残します。

```
{
 "proposal": {
  "kind": "preference", "subject": "user",
  "content": "ユーザーは静かな部屋を好む。",
  "confidence": 0.9,
  "evidence_event_ids": ["live_example_fragment"],
  "retention": "long"
 },
 "evidence": [
  { "quote": "私は静かな部屋が好きです。",
    "evidence_event_ids": ["live_example_fragment"] }
 ],
 "explicit": true,
 "conflict": false
}
```

別の照合依頼で本文が支持され、コードの条件も通って初めて自動採用の対象になります。保存レコードにはid、status、source、session_id、created_at、source_key、照合状態などが追加されます。候補の段階のJSONと、採用済みストアの全形式を混同しないでください。

ID	指しているもの	主な対応関係
observation id	一つの観測	trigger.idや根拠IDになる
turn_id	一回の判断依頼	AI応答が同じ値をコピー
Live session id	一つの音声接続	会話断片と委譲を束ねる
delegation id	Liveからの一つの委譲	commentaryなどの返却先
intention id	許可された一つの実行意図	outcomeが参照する
memory id / source_key	記憶レコード / 適用の出所	検索・置換 / 再試行の重複防止

同じ「ID」でも用途は異なります。たとえばメモリーのsession_idはプロセス側の保存範囲を示し、Live session idと同じ値とは限りません。IDを一列に並べず、どの境界を結ぶかを意識すると障害を追いやすくなります。

根拠（作業ツリー）：src/live/memory-extractor.js:43 / src/memory/memory-store.js:111 / src/live/conversation-store.js:44 / src/behavior/action-gate.js:63

60 なぜこの部品が必要か：設計意図の対応表

実装が増えると、関数やファイルの名前を知っていても、何のためにあるのかが分からなくなります。現在の設計を、解決したい具体的な問題へ結び直した表です。理由は、コードの責任分割と失敗時の動きから読み取れる説明です。

起きる問題	それを扱う部品	その部品がある意味
入力元ごとに形式が違う	observationと各正規化	同じ順序・由来の仕組みへ載せる
古い測定を今の事実にしてしまう	StateStoreの鮮度	過去値と利用可否を分ける
数値の解釈が会話でぶれる	WorldModelの規則	閾値と根拠を再現可能にする
身体の状態が会話に効かない	DriveSystemとbodyContext	必要度や事実を判断・会話へつなぐ
操作待ちで会話が止まる	Liveとバックエンドの分割	会話と仕事の進行を並行させる
古い返答やJSONを誤実行する	TurnRegistryと形式監査	現在の依頼に対応した形だけ通す
同じ操作を何度も行う	ActionGate	内容・種類・ターンの制限を課す
まだ成功していないのに成功と言う	intention / outcome	予定と確認結果を分ける
会話から架空の事実を記憶する	原文根拠と二段階照合	抽出と採用を分け、検証材料を残す
保存途中の再試行で重複する	prepared / sourceKey / commit	同じ判断を再適用して位置を進める
再起動で何をしたか分からない	観測ログと復旧	未完了を成功と推測せず明示する
黙って失敗して理解できない	ダッシュボードと診断	各段階の状態を人が確認できる

読み進めたあとの見方

各部品は、自然な会話、説明可能な状態、慎重な操作、次回へ続く記憶という異なる要求をつないでいます。一つの大きなAIがすべてを自由に決めるのでなく、どの情報が事実で、どの提案を実行し、何を結果として残すかをコードの境界で整理しています。

今後の変更を見るときも、新しい機能名だけを追うより、この表のどの問題を解くのか、どの境界を新たに通るのかを確かめると、現在の構造とつながります。

根拠（作業ツリー）：src/behavior/index.js / src/behavior/action-gate.js / src/live/kernel-bridge.js / src/live/memory-extractor.js / src/behavior/event-store.js